

Reasoning with Large Language Models, a Survey

Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens,
Niki van Stein, Thomas Bäck
LIACS, Leiden University,
Netherlands

July 17, 2024

Abstract

Scaling up language models to billions of parameters has opened up possibilities for in-context learning, allowing instruction tuning and few-shot learning on tasks that the model was not specifically trained for. This has achieved breakthrough performance on language tasks such as translation, summarization, and question-answering. Furthermore, in addition to these associative “System 1” tasks, recent advances in Chain-of-thought prompt learning have demonstrated strong “System 2” reasoning abilities, answering a question in the field of artificial general intelligence whether LLMs can reason.

The field started with the question whether LLMs can solve grade school math word problems. This paper reviews the rapidly expanding field of prompt-based reasoning with LLMs. Our taxonomy identifies different ways to generate, evaluate, and control multi-step reasoning. We provide an in-depth coverage of core approaches and open problems, and we propose a research agenda for the near future. Finally, we highlight the relation between reasoning and prompt-based learning, and we discuss the relation between reasoning, sequential decision processes, and reinforcement learning. We find that self-improvement, self-reflection, and some metacognitive abilities of the reasoning processes are possible through the judicious use of prompts. True self-improvement and self-reasoning, to go from reasoning *with* LLMs to reasoning *by* LLMs, remains future work.

1 Introduction

Transformer-based Large Language Models (LLMs) that are trained on large datasets have achieved breakthrough performance at next token prediction [Vaswani et al., 2017, Radford et al., 2019, Wei et al., 2022a]; they are very good at natural language understanding (GLUE, SQUAD, Xsum) [Wang et al., 2018, 2019, Rajpurkar et al., 2016, Narayan et al., 2018], translation [Kocmi et al., 2022, Papineni et al., 2002, Sennrich et al., 2015], question answering [Tan et al., 2023], and other *System 1* tasks [Kahne-

man, 2011].¹ The success of ChatGPT [Ouyang et al., 2022] has taken the world by storm.

Transformer-based generative language models whose size is beyond hundreds of billions parameters are not only very good at language generation, they also enable new type of machine learning, called *in-context learning* [Brown et al., 2020]. In-context learning, also known as prompt-based learning, occurs only in LLMs beyond a certain size (hundreds of billions of parameters) that are sufficiently rich [Wei et al., 2022a]. In-context learning is inference time, prompt-based, few-shot learning, where model parameters are not trained or fine-tuned.

System 1 tasks, such as associative language tasks, are easily solved by LLMs with prompt-based learning, as the many school children around the world that use ChatGPT daily can attest. (Although the problems are too often not solved *correctly*, just *fluently*, when the model’s association powers lead to hallucination [Huang et al., 2023].) On the other hand, System 2 tasks, such as grade school math word problems, are more difficult for LLMs [Cobbe et al., 2021]. To solve math word problems we need to break down the problem in multiple reasoning steps. Spurred-on by the impressive performance on System 1 tasks, much research has focused on understanding the reason for the poor performance of LLMs on System 2 tasks, and how it can be improved.

Among this research, the Chain-of-thought experiment [Wei et al., 2022b] stands out. This work, and subsequently Kojima et al. [2022], showed that adding a simple instruction to the prompts, *Let’s think step by step*, can provoke an LLM to perform the required intermediate reasoning steps, achieving a surprising jump in performance. The Chain-of-thought paper is a breakthrough in the field of reasoning with LLMs. Much exciting work has been published that builds on this work.

Grade school math word problems started the research into LLM-reasoning, with the GSM8K benchmark [Cobbe et al., 2021]. In our survey we discuss papers based on this benchmark, and directly-related follow up work on reasoning. We focus on prompt-based approaches. We survey the recent literature using a straightforward taxonomy.

Although the field has only recently started, the jump in performance on reasoning has excited artificial intelligence and society alike. We provide a research agenda with opportunities for future research. At the end of this survey, we also discuss connections to other fields, such as self-reflection, metacognition (or thinking about thinking, see for example Dunlosky and Metcalfe [2008]), and the motivation towards artificial general intelligence.

Our contributions are:

- A survey of relevant approaches in prompt-based reasoning (grade school math word problems and closely related domains) in large language models, including a research agenda.

¹In his book *Thinking, fast and slow*, a bestseller on human psychology, Daniel Kahneman described System 1 thinking as a near-instantaneous process; it happens automatically, intuitively, and with little effort. It is driven by instinct and experiences. System 2 thinking is slower and requires more effort. It is conscious and logical. The automatic operations of System 1 generate surprisingly complex patterns of ideas, but only the slower System 2 can construct thoughts in an orderly series of steps. In the LLM literature the terms are often used as shorthand to distinguish single-step associative tasks, from multi-step reasoning tasks, despite the fact that language tasks such as question answering and translation may require some “slow” thinking.

- A taxonomy based on regular reasoning literature (step generation, step evaluation, and control of reasoning steps).

This survey is organized as follows. Section 2 summarizes the most relevant developments in LLMs, including in-context learning. Of great importance are the benchmarks that are used in this field. We discuss these in Section 3, followed by our method for scoping and selecting of papers in Section 4. Next, in Section 5 we provide a taxonomy of the field, where we discuss the approaches in detail. Then, in Section 6 we discuss our findings in a broader perspective. We also discuss the relation between reasoning and work on self-reflection and metacognition. This section concludes with an agenda for future research. Finally, Section 7 concludes the survey.

2 Background: Reasoning with LLMs

Before we dive into the works on reasoning, we review some background terminology on LLMs. Our overview is brief. Excellent surveys on LLMs are, for example, Minaee et al. [2024] and Zhao et al. [2023]. We discuss the generic training pipeline for LLMs, we discuss how in-context learning works, and we discuss the reasoning pipeline. We start with the generic language model training pipeline.

2.1 Training Pipeline Language Model

LLMs are typically constructed in a sequence of stages, from data preparation, through training, to inference. The training pipeline for most LLMs is quite elaborate. We will now list a pipeline of the most common stages, based on the survey by Minaee et al. [2024].

1. *Acquire* a large, general, unlabeled, high-quality text corpus. Some considerations on the selection of the texts are discussed in Brown et al. [2020].
2. *Pretrain* the transformer model [Vaswani et al., 2017] on this large corpus. This step yields a generalist model. The pretraining is done using a self-supervised approach on the unlabeled dataset (text corpus).
3. *Finetune* the general model to a specific (narrow) task. This can be done using supervised-learning with a new labeled dataset consisting of prompts and answers (supervised finetuning, SFT) [Wei et al., 2022a, Minaee et al., 2024], specific for the task at hand. (A small number of papers in this survey work in the finetuning stage.)
4. *Instruction tuning* is a form of finetuning on a labeled dataset of instruction prompts and corresponding outputs, to improve instruction following, and thus the usefulness of models.
5. *Align* the finetuned model with user expectations (preference alignment). The goal of this stage is to improve the model to give more ethically and socially acceptable answers. The machine learning method that is used in this stage can

be, for example, Reinforcement Learning with Human Feedback [Ouyang et al., 2022] or Direct Preference Optimization [Rafailov et al., 2024].

6. *Optimize* training to improve cost-effectiveness, for example, with low-rank optimization [Hu et al., 2021], mixed precision training [Micikevicius et al., 2017], quantization [Jacob et al., 2018], or knowledge distillation [Xu et al., 2024, Gu et al., 2023].
7. *Inference & In-context learning* can be used to train the model to provide the correct answers without changing parameters [Dong et al., 2022, Brown et al., 2020]. By providing a prompt that contains a small number of examples together with a question, prompt learning is a form of few-shot learning. This is the stage in which most of the papers of this survey work, and that is familiar to all general users of ChatGPT.

Most of the reasoning methods that we discuss in this survey work in stage 7: in-context learning, using prompts for the LLM to perform a complex multi-step reasoning task. The following section provides a brief introduction to in-context learning.

2.2 In-Context Learning

In LLMs beyond hundreds of billions of parameters a new kind of learning has emerged, that is called *in-context learning* or prompt-learning [Brown et al., 2020]. It occurs at inference time, and is often able to give good results with few examples; it is a form of few-shot learning. The large size of the model, containing rich and general knowledge is enabling this new type of few-shot learning (see Dong et al. [2022] for a survey).

A prompt, consisting of a piece of demonstration context, is concatenated with a query question, and is given to the language model for prediction [Liu et al., 2023]. For example, when the task is emotion recognition in a social media post, “I missed the bus today,” can be followed by “I felt so [___]”. Alternatively, for translation, we could follow “I missed the bus today,” by “French: [___]” [Liu et al., 2023]. The prompt contains background information that is recognized by the model, selecting the desired model context. In-context learning works when language models contain enough knowledge, allowing them to generalize on the examples provided in the prompt.

Prompts that contain a few examples are said to perform few-shot learning. Prompts that contain only instructions without examples are said to perform zero-shot learning.

In-context learning takes place at inference time, after the computationally intensive stages where parameters have been pretrained and finetuned, when the model is queried by the user to provide answers. No parameters are changed anymore with in-context learning. This is quite different from the common approach in supervised deep learning—or self-supervised deep learning—where large datasets are used during training to update model parameters with backward propagation in lengthy and costly training epochs [Goodfellow et al., 2016]. Common approaches to few-shot learning, such as metalearning, do include training and finetuning of parameters to achieve generalization, and are computationally expensive (see, for example, Finn et al. [2017] or Huisman et al. [2021], Hospedales et al. [2021] for a survey).

Prompts provide a user-friendly interface to LLMs. The success of in-context learning tends to be quite sensitive to the way a prompt is formulated; a new field called *prompt engineering* has emerged to optimize the usefulness of in-context learning by learning how to make them do what we want [Radford et al., 2019, Wei et al., 2022a, Giray, 2023, Sahoo et al., 2024].

2.3 Reasoning Pipeline

Reasoning problems are also solved with a pipeline of stages. A typical approach to solving a complex problem is to subdivide it into smaller steps and solve those. This approach is related to divide and conquer [Bellman, 1966]. New steps are (1) *generated*, (2) *evaluated*, and the number of steps that are generated and searched is (3) *controlled* in some way. The in-context reasoning approaches that we survey follow a general three-stage pipeline [Madaan et al., 2023]:

1. **Generate**: generation of steps by the model,
2. **Evaluate**: evaluation of the predicted steps by an evaluator,
3. **Control**: control of the number of steps that are generated and how deep ahead the reasoning process will look.

This three-stage pipeline will be the basis of our taxonomy. But first, we will look at benchmarks.

3 Benchmarks

Progress in artificial intelligence is measured by benchmarks. Benchmarks define the goal that researchers aim to achieve in their experiments. In natural language processing, a wide array of benchmarks exists to measure progress, such as on *question answering* (for example, CommonsenseQA [Talmor et al., 2018]), *word prediction* (for example, LAMBADA [Paperno et al., 2016]), *translation* (for example, WMT’22 [Kocmi et al., 2022]), *language understanding* (for example, GLUE [Wang et al., 2018, 2019]), and *text summarization* (for example, Xsum [Narayan et al., 2018]). Transformer architectures were first popularized by encoder models such as BERT [Devlin et al., 2018], for named entity recognition and classification tasks. Subsequently, decoder models such as GPT 2-4 [Radford et al., 2019, Brown et al., 2020, Achiam et al., 2023] showed impressive progress on natural language benchmarks.

The field of LLMs is quite active. Many different benchmarks exist, and listing a comprehensive overview of all relevant benchmarks is beyond the scope of this survey. We will mention relevant benchmarks for testing the reasoning abilities of LLMs. Following Wei et al. [2022b], these are all math word problem benchmarks. The benchmark that is most frequently associated with reasoning by LLMs is a dataset of grade school math word problems GSM8K [Cobbe et al., 2021]. GSM8K was created by humans, with an aim of high quality, high diversity, moderate difficulty, and solutions in natural language. Other benchmarks are the SVAMP varying structures benchmarks [Patel et al., 2021], the ASDiv dataset of diverse math problems [Miao et al., 2021],

the AQuA dataset of algebraic word problems [Ling et al., 2017], and the MAWPS benchmark [Koncel-Kedziorski et al., 2016].

We will now briefly discuss these benchmarks; the baseline performance that we quote is from Wei et al. [2022b].

GSM8K To test reasoning skills, the Grade School Math problem dataset (GSM8K) was developed for testing LLMs [Cobbe et al., 2021]. It consists of 8500 human-written math problems. Language models struggled to achieve good performance on this dataset (pre Chain-of-thought). An example of a math word task is:

Problem: Beth bakes 4, two dozen batches of cookies in a week. If these cookies are shared amongst 16 people equally, how many cookies does each person consume?

Answer: $4 \times 2 \times 12/16 = 6$.

The baseline performance of GPT-3 175B is 15.6% accuracy. In comparison, the performance of Chain-of-thought is 46.9% accuracy.

ASDiv The Academia Sinica Diverse MWP Dataset (ASDiv) [Miao et al., 2021] is specifically designed for high diversity in problem types, formats and difficulty levels. It consists of 2305 problems. An example problem is:

Problem: A sandwich is priced at 0.75. A cup of pudding is priced at 0.25. Tim bought 2 sandwiches and 4 cups of pudding. How much money should Tim pay?

Answer: $0.75 \times 2 + 0.25 \times 4 = 2.5$.

The baseline performance of GPT-3 175B is 70.3% accuracy. The performance of Chain-of-thought is 71.3% accuracy.

MAWPS The Math Word Problem Repository (MAWPS) [Koncel-Kedziorski et al., 2016] allows for the construction of datasets with particular characteristics by selecting different categories of problems. The dataset consists of 3320 problems. An example is:

Problem: Rachel bought two coloring books. One had 23 pictures and the other had 32. After one week she had colored 44 of the pictures. How many pictures does she still have to color?

Answer: $55 - 44 = 11$.

The baseline performance of GPT-3 175B is 72.7% accuracy. The performance of Chain-of-thought is 87.1% accuracy.

SVAMP The Simple Variations on Arithmetic Math word Problems dataset (SVAMP) was designed by Patel et al. [2021]. It consists of 1000 problems, curated from variations of ASDiv-a [Miao et al., 2021] and MAWPS [Koncel-Kedziorski et al., 2016]. An example problem is:

Problem: Jack had 8 pens and Mary had 5 pens. Jack gave 3 pens to Mary. How many pens does Jack have now?

Answer: $8 - 3 = 5$.

The baseline performance of GPT-3 175B is 65.7% accuracy. In comparison, the performance of Chain-of-thought is 68.9% accuracy.

AQuA The Algebraic Question Answering dataset [Ling et al., 2017] is a large dataset of 100,949 questions, answers, and rationales. The dataset is based on a combination of a smaller seed dataset and crowdsourcing. An example question is:

Question: Two trains running in opposite directions cross a man standing on the platform in 27 seconds and 17 seconds respectively and they cross each other in 23 seconds. The ratio of their speeds is: *Options:* A) $3/7$ B) $3/2$ C) $3/88$ D) $3/8$ E) $2/2$

Answer: B.

The baseline performance of GPT-3 175B is 24.8% accuracy. The performance of Chain-of-thought is 35.8% accuracy.

There is a wide variety of benchmarks, and there is a wide variety of performance in benchmarks. Some are easily solvable by current LLMs, and some are significantly harder. Benchmark design is an important part of the field of reasoning in LLMs. Currently the GSM8K benchmark is popular; baseline model performance is weak, and reasoning prompts can substantially improve performance. As performance on GSM8K improves, different (harder) benchmarks will become popular.

4 Selection of Papers

The papers in this survey were selected as follows. Baseline LLMs have difficulty solving math word problems, specifically on benchmarks listed in the previous section. We take the ability to solve those benchmarks as a proxy for *reasoning* ability. We initially performed a literature search for papers that use these benchmarks, and that contain the search terms *reasoning* and *large language model* in their title or abstract. We also searched for papers that referenced the Chain-of-thought paper. The resulting papers were curated based on recency, relevance, substance, and novelty.

We favor recent papers (two years prior to the writing of the survey), related to the Chain-of-thought approach of generating intermediate reasoning steps, that solve tasks such as math word problems, and that work by prompt-based in-context learning. We also include some papers that work by finetuning or supervised learning that relate to, or inspire, the Chain-of-thought approaches. Furthermore, we include approaches outside math word problems that showed interesting approaches to reasoning, such as applications in coding and autonomous agents, because of their approach to grounding.

5 Prompt Generation, Evaluation and Control

This survey examines how an architecture that is good at System 1 tasks can be prompted to solve System 2 tasks. The Chain-of-thought paper showed how a simple command could prompt an LLM to perform reasoning steps, yielding much better performance in math word problems. Since then much research has further explored this approach, trying to build the ultimate general problem solver for System 1 and System 2 problems.

Following the pipeline of Section 2.3, the prompts must (1) generate the reasoning steps, (2) evaluate the answer to the steps, and (3) control the number of steps that are generated, the shape (or complexity) of the reasoning process must be controlled. We will now briefly discuss the three stages. Please refer to Figure 1 for a diagram of the different approaches for the generation, evaluation, and control of reasoning steps, and to Table 1.²

Prompt for Step Generation The first order of business is to create a prompt that instructs the LLM to generate reasoning steps. The problem must be split into substeps. This can be achieved with a problem-specific prompt that contains elements of the problem, such as: “First calculate how many marbles Mary had originally, then how many her friend had, and finally how many they had together.”

In general, it is possible to prompt an LLM to fill in the blanks in a step-by-step fashion. In the papers that we will discuss, there are three main approaches for generating the step-by-step prompt. The prompt may be (1) handcrafted for the problem by the researchers (*hand-written prompt*), or (2) the prompt or prompts may come from an source that is external to the model, such as another model or a dataset (*prompt using external knowledge*), or (3) the model itself can be prompted to generate a (series of) prompt(s) to analyze the problem (*model-generated prompt*). As we will see, all three approaches have their advantages and disadvantages.

Generating the subproblem-steps is the first stage that is necessary for in-context learning to perform reasoning. Each paper in our survey performs at least this stage of the reasoning pipeline. In some of the early papers (around 2022) it is the only stage of the pipeline that is performed.

Prompt for Result Evaluation After the prompt has been generated and the model has answered it, the next step in the reasoning pipeline is to evaluate the answer. Again, we see three main approaches for substep evaluation. First, the steps may be evaluated by (1) the model itself (*self-assessment*). Second, (2) an external program can be used to evaluate the steps. For example, when the steps are expressed as computer code, an external interpreter or compiler can be used to check the validity and the outcome (*tool-based evaluation*). Finally, (3) an external model can be used, LLM or otherwise. For example, in robotics, an external physics model can determine if certain actions are physically possible (*external model validation*).

²We show the approaches in the Figure in their main category only. Some approaches show innovations in two categories, and are shown twice. (Since all approaches have a generation, an evaluation, and a control aspect, all could in principle occur three times—all three columns can be found in Table 1).

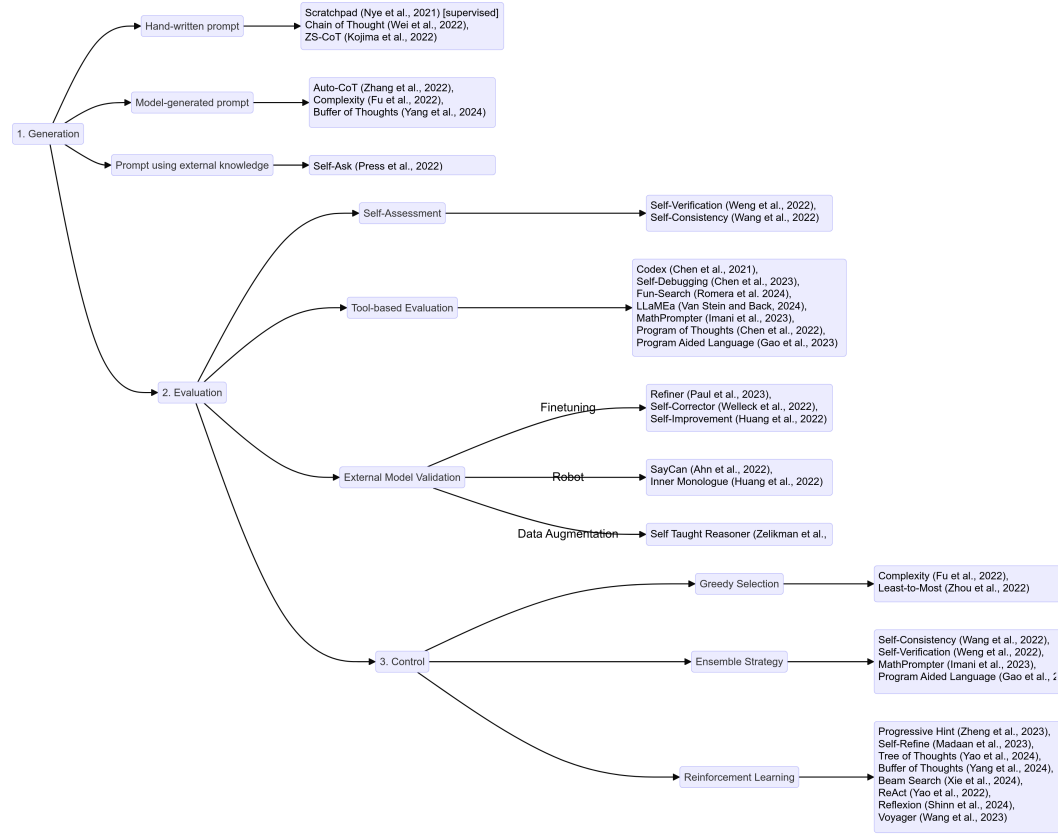


Figure 1: Taxonomy of LLM-Reasoning Approaches: Prompt Generation, Evaluation, and Control

Perform Control of Reasoning Steps A reasoning process that consists of multiple steps is a sequential decision process [Littman, 1996]. When a single chain of reasoning steps is generated, the control flow of the reasoning process is simple: greedily evaluate the first step and then the next one, if present. The control flow of the reasoning process may also be more intricate. Some reasoning problems can be divided into multiple subproblems. To execute, evaluate and combine the results of all substeps, a separate controller may be needed. This controller can be a prompt or an external algorithm.

Again, we distinguish three approaches. Most papers use (1) a *greedy selection approach*: a single prompt with a single chain of steps is generated, and these steps are directly executed and followed. The second approach (2) is to generate an *ensemble strategy* of reasoning steps, evaluate them, combine the individual results, and present them as the result of the ensemble. Finally, (3) a *full tree-search or a reinforcement learning (RL)* algorithm can be used as scaffolding. In this case, when a step is followed and evaluated, the LLM can roll back and try a different reasoning step. This is a breadth-first search approach [Plaat, 2020]. Going further, a full reinforcement learning approach can be used [Sutton and Barto, 2018, Plaat, 2022] to find an optimal policy for the sequential decision process. A full Markov Decision Process of state, action, transition, and reward function is specified, and step control can become a process where prompts are generated dynamically.

Domain Many papers are applied to math word problems (natural language descriptions of math problems). Math problems were the original inspiration for the experiments with reasoning in LLMs. Other application domains include autonomous agents, robotic movement, generating computer programs, and playing computer games. We will discuss these in more detail with the individual approaches.

Taxonomy Table Table 1 lists the papers of this survey. They are listed by the domain they work on, the type of prompt generation, the evaluation of the result, and the control method. The approaches in the table are grouped, divided by horizontal lines.

The first group, from Scratchpad to Self-ask, focuses on creating a prompt that generates the reasoning steps. The entries in the cells of this column are shown in bold, highlighting the focus of the approaches. The approaches in this group can be considered to be the start of the field of LLM-reasoning. The Chain-of-thought approach is especially an inspiration for many works. The prompts are often written “manually” by the researchers, the steps are encoded in one prompt, and step control is greedy. There is no specific evaluation of the steps, other than comparing results to the benchmark. The Scratchpad approach is special in that it uses supervised learning, not prompt-learning; the work showed that LLMs can be made to generate internal reasoning steps by supervised learning, paving the way for the later prompt-based papers.

The second group, from Self-verification to Self-taught-reasoner, focuses on evaluation of the reasoning steps in the prompt. This column is shown in bold in the table. The approaches in this group aim to improve the Chain-of-thought results by reducing the error accumulation that occurs when multiple steps are taken in a reasoning chain. A variety of step control methods is used by these approaches, which is dis-

Table 1: Taxonomy of approaches: Generation, Evaluation, and Control

Approach	Domain	Step generation	Step evaluation	Step control
Scratchpad [Nye et al., 2021]	math word	hand-wr/supervised	-	greedy/1 prompt
Chain-of-thought [Wei et al., 2022b]	math word	hand-written	-	greedy/1 prompt
ZS-CoT [Kojima et al., 2022]	math word	hand-written	-	greedy/1 prompt
Auto-CoT [Zhang et al., 2022]	math word	model-generated	-	clustering
Complexity [Fu et al., 2022]	math word	hand-written	self-consistency	greedy/1 prompt
Self-ask [Press et al., 2022]	math word	external knowledge	LLM	multi-hop questions
Self-verification [Weng et al., 2022]	math word	hand-written	back-verify	ensemble
Self-consistency [Wang et al., 2022b]	math word	hand-written	majority	ensemble
Codex [Chen et al., 2021]	code	-	tool-based	-
Self-debugging [Chen et al., 2023]	code	hand-written	tool-based	greedy
Fun-search [Romera-Paredes et al., 2024]	code	hand-written	tool-based	evolutionary algorithm
LLaMEa [van Stein and Bäck, 2024]	code	hand-written	tool-based	evolutionary algorithm
MathPrompter [Imani et al., 2023]	math	hand-written	tool-based	ensemble
Program-of-thoughts [Chen et al., 2022]	math word	hand-written, Codex	Python+Consist.	decouple reason/compute
Program-aided-language [Gao et al., 2023]	math word	hand-written, Codex	NLP/Python	ensemble
Refiner [Paul et al., 2023]	math word	finetune	critic model	gen/crit feedback
Self-corrector [Welleck et al., 2022]	math word	finetune	corrector model	gen/corr feedback
Self-improvement [Huang et al., 2022a]	math word	finetune	self-assessment	CoT/consistency
Say-can [Ahn et al., 2022]	robot	model-generated	external model	greedy
Inner-monologue [Huang et al., 2022b]	robot	hand-written	various	greedy
Self-taught-reasoner [Zelikman et al., 2022]	math word	finetune	augmentation	greedy/feedback
Least-to-most [Zhou et al., 2022]	math word	hand-written	self-assessment	curriculum
Progressive-hint [Zheng et al., 2023]	math word	model-generated	self-assessment	stable prompt
Self-refine [Madaan et al., 2023]	math word	model-generated	self-assessment	greedy/feedback
Tree-of-thoughts [Yao et al., 2024]	puzzles	model-generated	self-assessment	BFS/DFS
Buffer-of-thoughts [Yang et al., 2024]	math word	thought template	self-assessment	buffer manager
Beam-search [Xie et al., 2024]	math word	model-generated	self-assessment	Beam Search
ReAct [Yao et al., 2022]	action	external knowledge	self-assessment	reinforcement learning
Reflexion [Shinn et al., 2024]	decision	model-generated	ext model	reinforcement learning
Voyager [Wang et al., 2023]	Minecraft	model-generated	Minecraft	reinforcement learning

cussed in more detail later. Note that not all approaches use natural language problems (often math word problems). For example, the subgroup of Codex to Program-aided-language focuses on formal languages. They generate code or math equations, typically in Python, to formalize the steps of the reasoning problem, or as result of the task. LLMs are quite good at code generation, and these approaches typically achieve good performance. The use of code also allows the approaches to call external programs such as interpreters and debuggers to evaluate the correctness of the reasoning steps that are generated.

There is also a special subgroup, Refiner to Self-improvement, that does not use prompt learning but finetuning. Here, new data is generated based on reasoning exemplars, which is then used to further train the model. The extra data is often generated as a separate dataset, sometimes called *critic* or *corrector*.

There are two approaches, Say-can and Inner-monologue, whose application domain is control of robot movement. Robotic movement is constrained by the laws of physics (both in the body of the robot as in aspects of its environment). The laws of physics are learned and used to ground the reasoning steps in reality (to reduce hallucination).

The third group, Least-to-most to Voyager, addresses step control (approaches shown in bold in this column). Whereas in the previous approaches the reasoning steps are written in a single, static, prompt, these approaches generate the steps in multiple, dynamic, prompts. This allows control of the space of reasoning steps. Various search control approaches are used, all in the form of an external algorithm that performs calls to the LLM with different prompts. The control methods range from simple greedy and depth-first search to elaborate beam search and reinforcement learning schemes.

In summary, we see a diverse array of methods that often achieve high performance in reasoning about their respective domains. To better understand the approaches, let us discuss the techniques in more detail, starting with the generation of steps.

5.1 Generation of Steps

Originally, LLMs performed poorly on math word problems (GSM8K [Cobbe et al., 2021]). Some different approaches were tried, for example scaling up the size of the LLM [Rae et al., 2021]. The LLM architecture, based on transformers, is designed to produce a single token. When we prompt such an architecture to produce an answer, it does so. What we should do is prompt it to follow intermediate steps, answer those, and thus work towards the final answer, just as a student is taught to break down a complex problem into smaller steps. We should take the model by its hand and teach it to write down the intermediate steps, and combine the intermediate results [Nye et al., 2021].

This idea was used by Nye et al. [2021] in Scratchpads, a transformer model that performs multi-step computations by asking it to emit intermediate computation steps into a *scratchpad*. They train the model by supervised learning (not prompt-based in-context learning). Figure 2 shows an example. On experiments with addition, polynomial evaluation, and Python code execution, versions that produced the intermediate steps on a scratchpad performed considerably better than versions that did not.

```

Input:
2 9 + 5 7

Target:
<scratch>
2 9 + 5 7 , C: 0
2 + 5 , 6 C: 1 # added 9 + 7 = 6 carry 1
, 8 6 C: 0 # added 2 + 5 + 1 = 8 carry 0
0 8 6
</scratch>
8 6

```

Figure 2: Example of input and target for supervised learning on a *long addition* problem of adding two numbers. The carry is recorded in the C: digit. Comments (after #) are not part of the learning target [Nye et al., 2021]

If supervised learning can produce intermediate steps, would prompt-learning be able to do so too?

5.1.1 Hand-written Prompt

This question was studied by Wei et al. [2022b], amongst others. A basic way to instruct an LLM to generate steps by prompt-learning is to manually write a prompt for the large language model to follow the reasoning steps. They showed in their Chain-of-thought paper that with such a prompt the LLM follows such intermediate steps. When the LLM is prompted to rephrase information from the question as intermediate reasoning steps in its answer, the LLM performed much better than when it was prompted to answer a math problem directly, without reproducing the information from the question in its answer. The example from the Chain-of-thought paper is shown in Figure 3 Wei et al. [2022b]. Performance figures were given in Section 3 on benchmarks.

The substantial performance improvement by Chain-of-thought has caused much excitement and has opened up further research on reasoning with LLMs. In the original Chain-of-thought paper the prompts were handwritten by the researchers for the individual types of problems, and evaluations are conducted with five different benchmarks (not by an LLM).³ In a later work the prompts were generated automatically by the LLM [Zhang et al., 2022].

Kojima et al. [2022] go a step further. They show that the simple addition of a single text to the prompt (*Let's think step by step*) significantly improves performance. Since this text does not contain problem-related elements, this can be considered as a form of *zero-shot learning*. Figure 4 compares the approaches. Experiments further show that with this addition to the prompt, significant performance gains are achieved

³The Chain-of-thought idea is about prompt generation, not about the evaluation or the search control of the reasoning steps. Hence, in Table 1 Chain-of-thought is labeled as *greedy* without an evaluation.

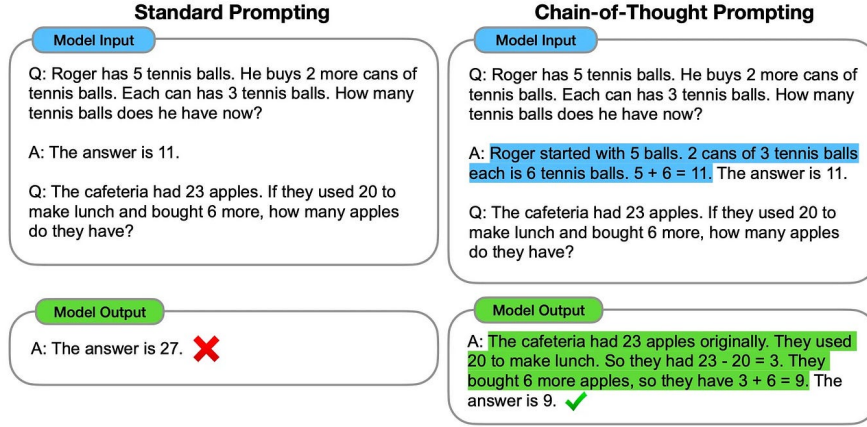


Figure 3: Chain of Thought Prompting. In blue at the top the prompt, in green at the bottom the answer. When shown the longer example prompt, the LLM follows the longer example when answering the question [Wei et al., 2022b].

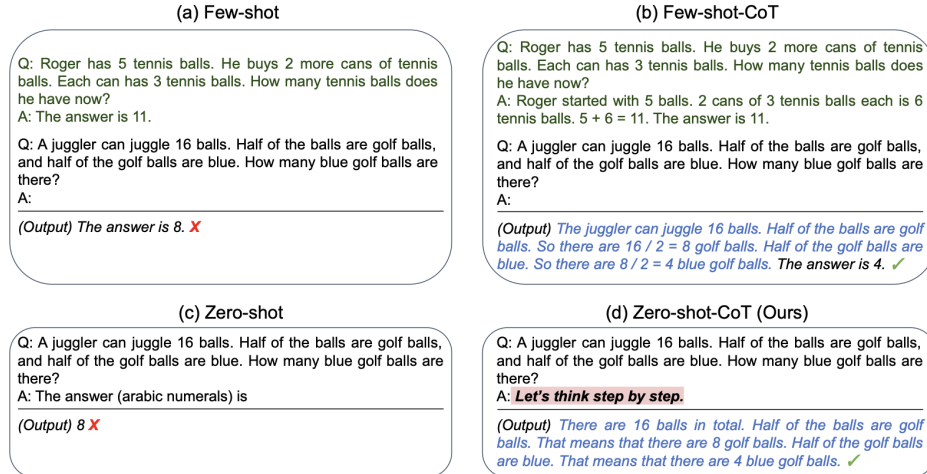


Figure 4: Zero-shot Chain-of-thought: *Let's think step by step* [Kojima et al., 2022]

Direct Prompting

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?
Answer: Harry Vaughan Watkins.

Question: Who was president of the U.S. when superconductivity was discovered?
Answer: Franklin D. Roosevelt



Chain of Thought

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?
Answer: Theodor Haecker was 65 years old when he died. Harry Vaughan Watkins was 69 years old when he died.
So the final answer (the name of the person) is: Harry Vaughan Watkins.

Question: Who was president of the U.S. when superconductivity was discovered?
Answer: Superconductivity was discovered in 1911 by Heike Kamerlingh Onnes. Woodrow Wilson was president of the United States from 1913 to 1921. So the final answer (the name of the president) is: Woodrow Wilson.



Self-Ask

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?
Are follow up questions needed here: Yes.
Follow up: How old was Theodor Haecker when he died?
Intermediate answer: Theodor Haecker was 65 years old when he died.
Follow up: How old was Harry Vaughan Watkins when he died?
Intermediate answer: Harry Vaughan Watkins was 69 years old when he died.
So the final answer is: Harry Vaughan Watkins

Question: Who was president of the U.S. when superconductivity was discovered?
Are follow up questions needed here: Yes.
Follow up: When was superconductivity discovered?
Intermediate answer: Superconductivity was discovered in 1911.
Follow up: Who was president of the U.S. in 1911?
Intermediate answer: William Howard Taft.
So the final answer is: William Howard Taft.



Figure 5: Self-Ask asks follow-up questions, and uses an external search engine [Press et al., 2022]

on a diverse set of reasoning benchmarks, including arithmetic, symbolic, and logical reasoning.

The Chain-of-thought idea itself is inspired by earlier work where natural language steps are generated for arithmetic reasoning [Ling et al., 2017, Cobbe et al., 2021], and the use of formal languages for reasoning [Roy and Roth, 2016, Chiang and Chen, 2018, Amini et al., 2019, Chen et al., 2019].

5.1.2 Prompt using External Knowledge

Chain-of-thought shows that an LLM gives better answers to complex problems when it is guided to take individual steps. Prompts are written manually, from scratch, by the researchers.

We can use external information about the problem to improve the prompt. Press et al. [2022] study how subproblems are related to the main problem, which they call *compositional reasoning*. They study how often a model is able to answer the subproblems, but not the overall problem. This difference is called the compositionality gap. They find that in GPT-3, as model size increases, the compositionality gap does not decrease: the single-hop question-answering performance improves faster than the multi-hop performance. This shows that while more powerful models memorize and recall more factual knowledge, no improvement in their ability to perform compositional reasoning occurs. They find that the ability to reason does not depend on the size of the model.

Subsequently, a method called *Self-ask* is proposed, that asks elicitive follow-up

questions (like Chain-of-thought, but with the *follow up:* prompt), see Figure 5. The model is then used to answer these follow-up questions. Self-ask can also use an external search engine to answer intermediate prompts, instead of the model. The model takes as input a compositional question which it decomposes. The initial subquestion is fed into the search engine, and the answer is processed by the model, which generates another subquestion, and so on, until it produces the final answer.

The approach performs a few percentage points better than vanilla Chain-of-thought on three benchmarks that were specifically designed for multi-hop questions.

5.1.3 Model-Generated Prompt

In addition to manually writing prompts or using external information, we can also try to let the LLM itself study the problem to write the best reasoning-prompt, a form of self-improvement. An example of this approach is Auto-chain-of-thought [Zhang et al., 2022]. This approach builds on the observation by Kojima et al. [2022] that large language models are zero-shot reasoners. First, Auto-chain generates specific questions for a given dataset and partitions them into clusters. Then an external algorithm uses the model to generate examples that are sampled for diversity. The constructed demonstrations augment the in-context prompt. The automatically generated prompts are reported to perform as well or better than the hand-written Chain-of-thought prompts on ten benchmarks using GPT-3.

Fu et al. [2022] introduce Complexity-based prompting. Inspired by Chain-of-thought and Self-consistency, this work studies which prompts achieve the best results on math word and other reasoning problems. Their work specifically studies the impact of the complexity of the reasoning chain, and introduces a related reasoning approach (Complexity-based prompting). They find that prompts with the largest complexity (the most reasoning steps) perform best. Further, they find that outputs (answers) with the highest complexity are the best. Complexity-based prompting achieves high performance on three math reasoning benchmarks.

Another approach that uses model-generated prompts is Buffer-of-thoughts. We will discuss this approach in Section 5.3.3.

5.2 Evaluation of Steps

After discussing prompts for the generation of reasoning steps, the next stage in the reasoning pipeline (Section 2.3) is *evaluation* of the results of the steps, to reduce the error of multi-step reasoning chains.

We will start with approaches where the same model performs step-generation and step-evaluation.

5.2.1 Self-Assessment

When LLMs are prompted to perform reasoning steps, they perform a sequence of steps and predict multiple tokens. Performing a sequence of steps makes them sensitive to mistakes and vulnerable to error accumulation [Weng et al., 2022, Xiao et al., 2023a]. Several methods have been developed to prevent error accumulation. One approach is

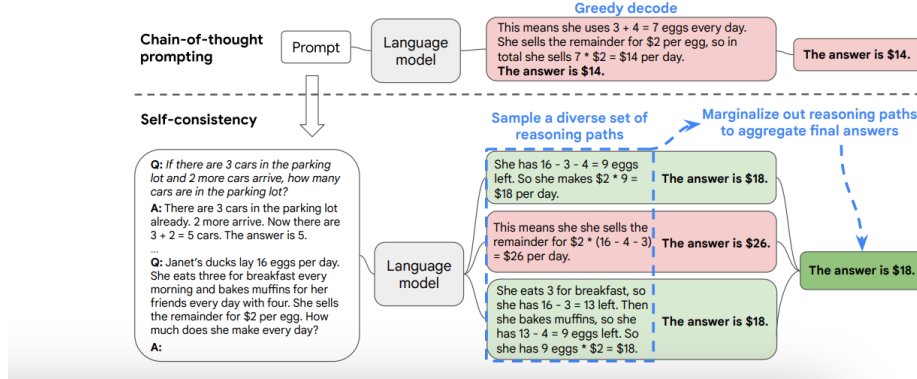


Figure 6: Self-Consistency [Wang et al., 2022b]

to create a new model to separately evaluate the results. Shen et al. [2021] and Li et al. [2022b] train an external verifier to check results.

In contrast, Weng et al. [2022] propose an automated approach using evaluation by the same LLM, called Self-verification. They note that human reasoning also suffers from the problem of accumulating errors, and that in human reasoning we frequently revisit our thought process to verify the accuracy of our reasoning steps. Thus, they propose to apply such a backwards self-verification approach. The LLM is prompted to use the conclusion of the Chain-of-thought reasoning chain as a condition for solving the original problem and then compare the answer going back to the original question. The LLM is given variations of its own conclusion and is instructed to choose the one with the highest similarity to the original question. (Note that there can be feedback issues using an LLM to evaluate itself, for a discussion see Zheng et al. [2024].) Experiments are reported on GPT-3 [Chen et al., 2021] and on Instruct-GPT [Ouyang et al., 2022]. The performance of Chain-of-thought was improved by a few percentage points on arithmetic and general reasoning tasks.

A popular related approach is called Self-consistency [Wang et al., 2022b]. Self-consistency is a straightforward ensemble approach. Greedy single-path decoding is replaced by sampling diverse reasoning paths, evaluating them, and selecting the most consistent answer. Self-consistency asks the LLM to simply perform the same query multiple times, and takes the majority-vote of the answers. Self-consistency works since complex reasoning problems typically allow different reasoning paths that lead to the correct answer. Figure 6 summarizes the approach.

Self-consistency has been evaluated on arithmetic reasoning, commonsense reasoning and symbolic reasoning, on a variety of LLMs, including GPT-3 [Tay et al., 2022, Brown et al., 2020, Thoppilan et al., 2022, Chowdhery et al., 2023]. Self-consistency improves the performance of Chain-of-thought typically by 10-20 percentage points, and has been used as a baseline in many of the other approaches in this survey. (Self-verification also reports that performance is improved when used in combination with

Self-consistency [Wang et al., 2022b] and with Program-aided-language [Gao et al., 2023].)

5.2.2 Tool-based Validation

Another possibility to improve the accuracy of evaluating the reasoning steps is to switch from a natural to a formal language. The advantage of a formal language is that it is less ambiguous than a natural language. Examples are computer languages, such as Python, or mathematical equations. Using a formal language for reasoning is a popular approach, and we discuss seven papers. Many approaches generate the steps in Python, and the code can then be evaluated by a formal evaluator, such as a compiler, debugger, or interpreter.

LLMs have been quite successful in generating computer code from natural language prompts. Chen et al. [2021] introduced Codex, a GPT model that was trained on publicly available code in the repository GitHub. A production version of this work was introduced under the name GitHub Copilot. Codex is able to generate correct programs from descriptions in natural language, such as commentary strings. Figure 7 shows examples that are produced by Codex.

The work on Codex is used as a basis for further research on reasoning in LLMs.

Human programmers, when writing code, typically follow a cycle of writing some code, executing it to look for errors, and then using the feedback to improve the code. This same approach is followed in the Self-debugging work [Chen et al., 2023]. Self-debugging teaches a large language model to debug its generated program code via few-shot demonstrations. It follows the same steps of (1) code generation, (2) code execution, and (3) code explanation (see Figure 8).

Self-debugging is able, without human feedback on the code’s correctness or error messages, to identify mistakes in the code that was generated by itself from investigating the execution results. Self-debugging can also provide an explanation of the generated code in natural language. It achieves strong performance on text-to-SQL generation, C++-to-Python transcoding, and text-to-Python generation.

Several works use self-debugging to generate working code tuned for solving specific problems automatically, without human feedback. Romera-Paredes et al. [2024] introduced FunSearch, an approach that integrates formal methods and LLMs to enhance mathematical reasoning and code generation. FunSearch is capable of producing functionally correct programs that adhere to specified requirements. It uses a genetic algorithm approach with multiple populations of candidate solutions (programs), which are automatically evaluated (using tools depending on the problem specification). In addition to the problem specification in the form of an evaluate function, also an initial program is given to the LLM in the first prompt. After evaluating a number of generated programs from the starting prompt, a new prompt using ‘best-shot prompting’ is created in an iterative fashion, combining a selection of k sampled programs in a sorted list (ascending according to their evaluation score), and the LLM is requested to generate program $k + 1$. Another work leverages evolutionary computation methods to generate and optimize evolutionary algorithms [van Stein and Bäck, 2024]. This approach, LLaMEA (Large Language Model Evolutionary Algorithm), utilizes LLMs to design and optimize evolutionary algorithms. The approach uses LLMs to generate ini-

```
def incr_list(l: list):
    """Return list with elements incremented by 1.
    >>> incr_list([1, 2, 3])
    [2, 3, 4]
    >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
    [6, 4, 6, 3, 4, 4, 10, 1, 124]
    """
    return [i + 1 for i in l]
```

```
def solution(lst):
    """Given a non-empty list of integers, return the sum of all of the odd elements
    that are in even positions.

    Examples
    solution([5, 8, 7, 1]) ==>12
    solution([3, 3, 3, 3, 3]) ==>9
    solution([30, 13, 24, 321]) ==>0
    """
    return sum(lst[i] for i in range(0, len(lst)) if i % 2 == 0 and lst[i] % 2 == 1)
```

```
def encode_cyclic(s: str):
    """
    returns encoded string by cycling groups of three characters.
    """
    # split string to groups. Each of length 3.
    groups = [s[(3 * i):min((3 * i + 3), len(s))] for i in range((len(s) + 2) // 3)]
    # cycle elements in each group. Unless group has fewer elements than 3.
    groups = [(group[1:] + group[0]) if len(group) == 3 else group for group in groups]
    return "".join(groups)

def decode_cyclic(s: str):
    """
    takes as input string encoded with encode_cyclic function. Returns decoded string.
    """
    # split string to groups. Each of length 3.
    groups = [s[(3 * i):min((3 * i + 3), len(s))] for i in range((len(s) + 2) // 3)]
    # cycle elements in each group.
    groups = [(group[-1] + group[:-1]) if len(group) == 3 else group for group in groups]
    return "".join(groups)
```

Figure 7: Codex [Chen et al., 2021]

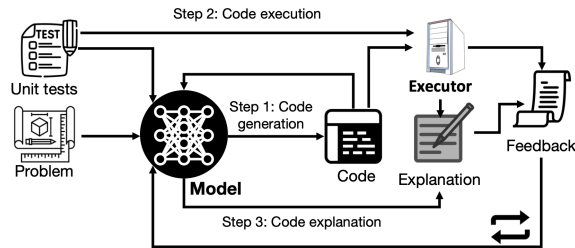


Figure 8: Self-Debugging [Chen et al., 2023]

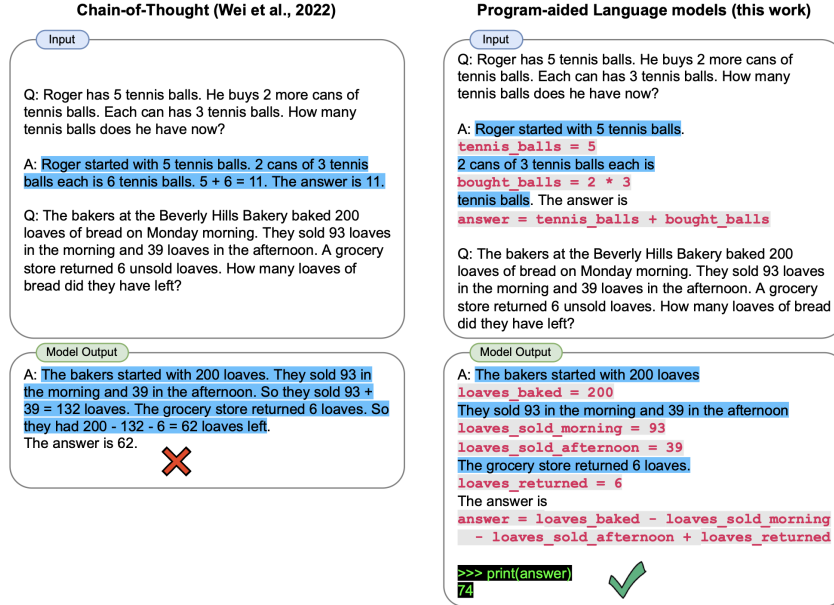


Figure 9: Program-aided-language [Gao et al., 2023]

tial algorithmic structures, which are then refined through mutation and selection. This enhances the efficiency of algorithm design, particularly in fields requiring innovative and adaptive solutions. A key difference between FunSearch and LLaMEA is that LLaMEA uses a sample-efficient elitism strategy by iteratively improving the best-so-far solution, requiring significantly fewer prompt evaluations than the large-population strategy proposed in FunSearch.

To improve prompt-based reasoning, Codex is used in an ensemble approach named MathPrompter [Imani et al., 2023]. This approach generates multiple algebraic expressions or Python functions, which then solve the same math problem. The results are compared, just like in Self-consistency and Self-verification, raising the confidence level in the results. MathPrompter achieved state-of-the-art results on the MultiArith dataset ($78.7\% \rightarrow 92.5\%$), evaluated on GPT-3 175B.

Two other approaches that use a formal language are Program-of-thought (PoT) [Chen et al., 2022] and Program-aided-language (PAL) [Gao et al., 2023]. Both approaches use the LLM to generate Python and then use a Python interpreter to evaluate the result. PoT and PAL are similar approaches. PoT uses benchmark-specific prompts; PAL uses generic prompts, and has been tested on more benchmarks and has been used in other approaches. Figure 9 illustrates the PAL approach.

When the evaluation of the reasoning steps is offloaded to the Python interpreter, decomposing the natural language problem into executable code-steps remains the only task for the LLM. (Earlier work in mathematical word problems, such as Ling et al. [2017], showed how to decompose a problem and reach an answer.) Gao et al.

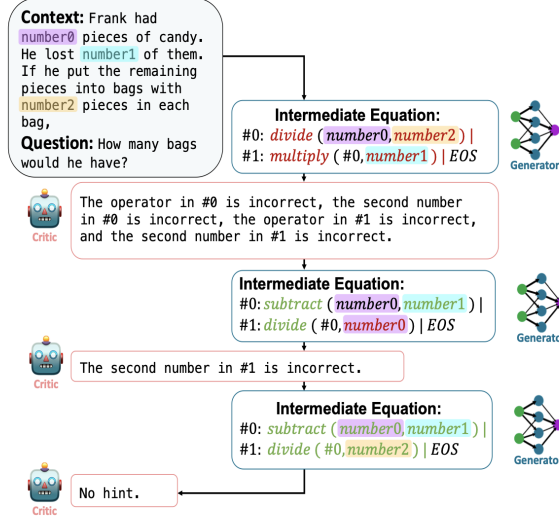


Figure 10: Refiner [Paul et al., 2023]

[2023] provide extensive experimental evidence about the synergy between the neural LLM and the symbolic interpreter. Experiments are performed over 13 mathematical, symbolic, and algorithmic reasoning tasks, achieving more accurate results than much larger models.

5.2.3 External Model Validation

We have seen many successful examples of prompt-based in-context reasoning and evaluation. We will now look at related reasoning approaches that follow a more traditional parameter learning approach. We describe three natural language approaches that follow this route. All approaches evaluate the output of the model and generate corrective data. That data is then added to the training pipeline, and the model is subsequently finetuned.

Finetuning The Refiner approach [Paul et al., 2023] uses a generator model and a critic model to provide fine-grained feedback on reasoning errors. The generator generates multiple reasoning hypotheses, the critic evaluates results by randomly selecting a hypothesis for feedback. The generator model is finetuned based on its reasoning errors. A small supervised model is used to overcome the cold-start problem. Figure 10 shows an example of how the critic provides feedback to the generator. The approach is reported to work well on math word problems and synthetic natural language reasoning.

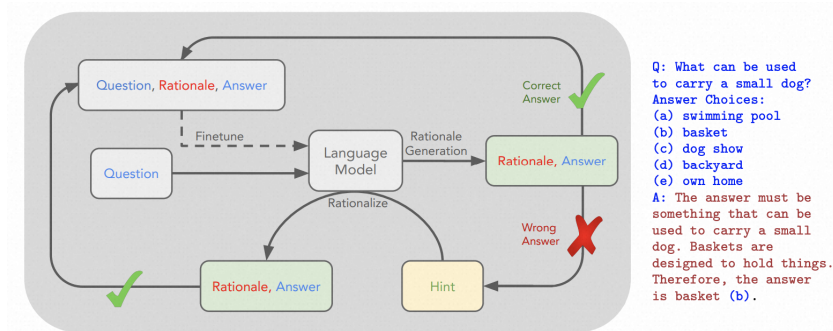


Figure 11: Self-Taught-Reasoner [Zelikman et al., 2022]

Welleck et al. [2022] follow a similar approach in their Self-correction approach. The corrector is a separate model specialized in refining the outputs of the generator. Unlike Refiner, where the generator is finetuned based on the critic feedback, Self-correction finetunes the corrector to rectify errors in the hypotheses produced by the generator.

A third finetuning approach is Self-improvement, by Huang et al. [2022a]. Here too the base model data is augmented by LLM-generated rationales, and then finetuned. Noteworthy in all three finetuning approaches is that LLMs are capable of improving themselves by training on their own generated output, and that stability problems inherent in feedback loops are overcome.

Dataset Augmentation The final finetuning approach that we discuss uses dataset augmentation. An explicit intermediate reasoning is called a *rationale*. Rationale generation has been shown to be valuable for LLMs across diverse tasks such as mathematical and commonsense reasoning, code evaluation, social bias inference, and natural language inference [Zelikman et al., 2022]. Zelikman et al. [2022] describe how reasoning steps are used to create rationales, that are then used to augment the dataset on which the model is finetuned. The approach is called Self-taught-reasoner. Figure 11 illustrates the approach. In Self-taught-reasoner, an augmentation dataset is created by attempting to solve the original dataset using the current model’s rationale generation ability in each iteration. Next, the dataset is augmented using rationalizations, using ground-truth answers to problems the model failed to solve. Finally, the large language model is finetuned on the combined dataset.

Reasoning about Robot Behavior In addition to math word problems, prompt-based reasoning has also been used to reason about robot behavior. Language models contain a large amount of information about the real world [Ahn et al., 2022]. In theory, this should allow the model to exhibit realistic reasoning about robotic behavior. However, the models do not have knowledge about particular embodied aspects of a particular robot. If we could compare a Scratchpad-like list of intermediate reasoning steps with



Figure 12: Say-Can compared to other language models [Ahn et al., 2022]

a list of possible movements of the robot in its environment, then we could prevent the model from suggesting impossible joint movements and actions, and prevent accidents.

Such an approach has been tried in the Say-can paper [Ahn et al., 2022]. Say-can learns a value function [Kaelbling et al., 1996] of the behavior of a robot in an environment using temporal difference reinforcement learning Sutton [1988]. This value function is then combined with prompt-based reasoning by the language model, to constrain it from suggesting impossible or harmful actions.

The goal of Say-can is to ground language in robotic affordances. In contrast to Scratchpad, which used supervised learning, the affordance model is learned interactively by reinforcement learning, and then applied using prompt-based learning by the LLM. The robot can act as the language model’s hands and eyes, while the language model has high-level semantic knowledge about the task. The LLM (Say) provides a task-grounding to find the actions to achieve the high-level goal. The learned affordance function (Can) provides a world-grounding to allow what is possible. Say-can is evaluated on 101 real-world robotic tasks, such as how to solve tasks in a kitchen environment (see Figure 12).

Where Say-can learns affordance as a separate function, another approach, Inner-monologue [Huang et al., 2022b] formulates robotic planning directly as part of the language prompt. This approach incorporates environmental information into the prompt, linguistically, as an inner monologue. As in Say-can, the information comes as feedback from different sources. Unlike Say-can, the information of physics and the world is inserted directly into the prompt.

Inner-monologue consists of many elements: it uses InstructGPT [Brown et al., 2020] for multi-step planning, scripted modules for object recognition, success detection, task-progress scene description, and language-conditioned pick-and-place primitives, similar to CLIPort [Shridhar et al., 2022]. These elements generate textual descriptions that are used in prompt-based learning. Figure 13 gives an example of the working of Inner-monologue.

The language feedback that is thus generated significantly improves performance on three domains, such as simulated and real table top rearrangement tasks and manipulation tasks in a kitchen environment. There are many studies into robotic behavior. A recent approach related to Inner-monologue is Chain-of-tools, which proposes a plan-

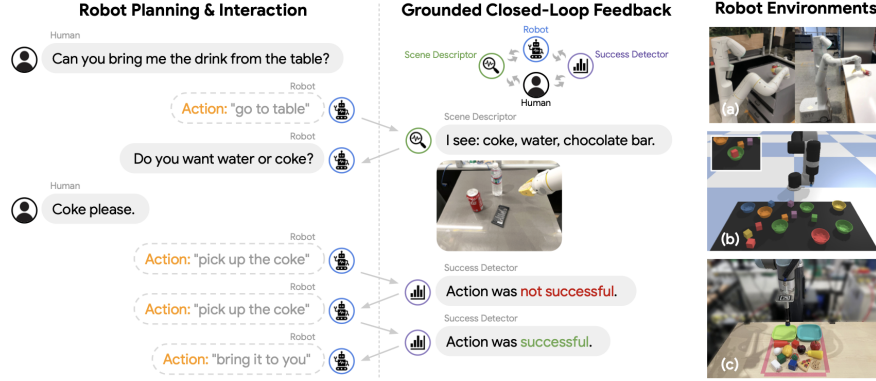


Figure 13: Inner-Monologue [Huang et al., 2022b]

execute-observe pipeline to ground reasoning about tool behavior [Shi et al., 2024a,b].

This concludes our discussion of the second stage of the reasoning pipeline, evaluation of the reasoning steps.

5.3 Control of Steps

The third stage in the reasoning pipeline in Section 2.3 is reasoning control. This stage controls how many sub-steps are generated, and how deep into the future the reasoning chain is generated.

There are three main approaches: (1) *greedy selection*, which generates a step and then follows it, (2) *ensemble strategy*, which generates a set of possible next steps, and (3) a full tree-shaped search which generates multiple options for the step, and follows them multiple steps into the future, traversing a search tree with backtracking, controlling an exponential search space. We include *reinforcement learning* approaches, that interactively learn an optimal policy for such a reasoning space.

5.3.1 Greedy Selection

Most earlier works on prompt-based reasoning follow the greedy approach: generate a single prompt with a sequence of steps and follow them. Among the greedy reasoners are Chain-of-thought, Auto-CoT, and Zero-shot CoT. Inner Monologue and Say-Can also use greedy reasoning.

In Least-to-most prompting [Zhou et al., 2022], the key idea is to break down a complex problem into simpler subproblems and then solve these in sequence, explicitly encoding them in the prompt. It is related to Complexity-based prompting. In Least-to-most, finding the answer to each subproblem is facilitated by the answers to previously solved subproblems, as in a curriculum [Bengio et al., 2009]. The authors find that on

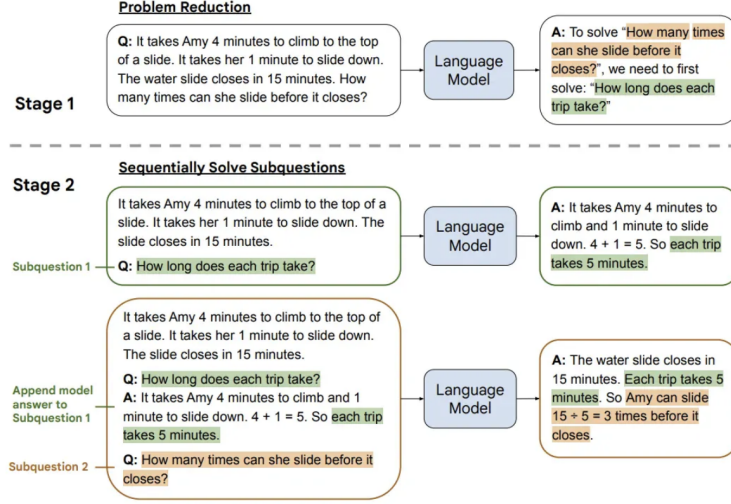


Figure 14: Least-to-most prompting [Zhou et al., 2022]

symbolic manipulation, compositional generalization, and math reasoning, the Least-to-most prompting is capable of generalizing to more difficult problems than those that are given in the prompts. Figure 14 illustrates the idea.

5.3.2 Ensemble Strategy

The second kind of reasoning control is based on an ensemble of (sequences of) reasoning steps. The ensemble approach is a well-known technique in machine learning to make a strong learner out of multiple weaker learners [Sagi and Rokach, 2018, Breiman, 2001]. For most problems, multiple different options for the next step exist. When all or some of these are generated and evaluated, then the best result or the consensus result can be reported as the outcome of an ensemble of steps. Various approaches have been proposed.

We already mentioned Self-consistency [Wang et al., 2022b] and Self-verification [Weng et al., 2022] in Section 5.2.1. They are popular ensemble approaches to evaluate the results of reasoning steps in prompt learning. The greedy single-path decoding used in Chain-of-thought prompting is replaced by sampling a diverse set of reasoning paths, evaluating them, and selecting the most consistent answer.

In another domain Chain-of-experts builds on Chain-of-thought with a mixture of experts ensemble for complex combinatorial operations research problems [Xiao et al., 2023b]. PAL and MathPrompter also use the ensemble approach. They generate multiple steps, which are evaluated and whose answer is combined, or the best step is chosen.

The ensemble approach is a popular approach in LLM-reasoning.

5.3.3 Reinforcement Learning

In the greedy approach, a single reasoning path is generated and traversed. In reasoning, often multiple valid reasoning steps are possible, but pursuing all possibilities over multiple reasoning steps may lead to an infeasible number of possibilities.

The third kind of reasoning control is to use a full-fledged controller that can traverse a tree, or even perform reinforcement learning to do so [Sutton and Barto, 2018, Kaelbling et al., 1996, Plaat, 2022]. This group of control approaches enables the most elaborate control of the reasoning process, and is used by many works, as we will see. When decomposing the problem, multiple alternative steps are generated that can be searched multiple steps into the future. Then, backtracking can be performed, allowing alternative steps to be tried.

Where greedy and ensemble processes can be controlled with a prompt by the LLM, this third group is more complex, and an external algorithm is used to control the reasoning process. The external algorithms call the LLM as a subroutine prompting it to perform its tasks. This allows more complex reasoning control, but we are no longer performing prompt-based self-reasoning; control has been given to an algorithm that is external to the LLM and external to prompt-learning.

We start our discussion of control strategies with depth-first and breadth-first search, then go to beam search, and then to full reinforcement learning.

Breadth first search A complex reasoning space can be traversed with a search algorithm. Tree-of-thoughts includes a search algorithm to dynamically follow different reasoning steps [Yao et al., 2024]. When one reasoning path has been traversed, a search algorithm can backtrack, and try an alternative path. The paper describes both a breadth-first-search and a depth-first-search controller.

The evaluation part in Tree-of-thoughts is performed with a prompt by the LLM. Together, the trio of generation, evaluation, and control allow systematic exploration of the space of reasoning steps with look-ahead and backtracking. The authors compare their approach to Chain-of-thought and Self-consistency. Chain-of-thought builds a reasoning out of a path of thoughts, Self-consistency creates an ensemble of thoughts, and Tree-of-thoughts constructs a tree structure. Figure 15 illustrates the different reasoning structures.⁴

Another approach, Buffer-of-thoughts [Yang et al., 2024], goes a step further towards meta-reasoning. It introduces a meta-buffer that stores high-level *thought-templates*. These universal thought-templates are derived from a variety of tasks. Figure 16 compares the Buffer-of-thoughts approach to other approaches such as Chain-of-thought and Tree-of-thoughts. Buffer-of-thoughts outperforms other methods in puzzles such as Game of 24 and checkmating. Thought templates are related to metacognition (thinking about thinking), which is further discussed in Section 6.2.3.

Beam search A related search method is Beam-search. Beam-search-for-reasoning [Xie et al., 2024] focuses on control of the space of possible reasoning paths. In some

⁴A similarly named approach is Graph-of-thoughts [Besta et al., 2024]. Graph-of-thoughts allows more general reasoning graphs, providing a formal framework, where the different elements can then be specified manually.

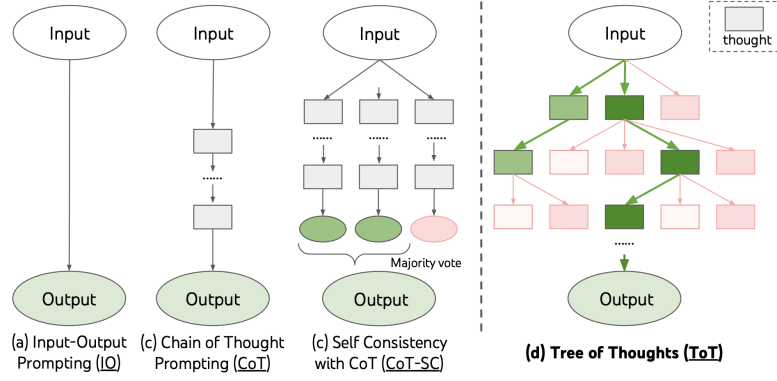


Figure 15: Reasoning structure of Chain-of-Thought, Self-Consistency, and Tree-of-Thoughts [Yao et al., 2024]

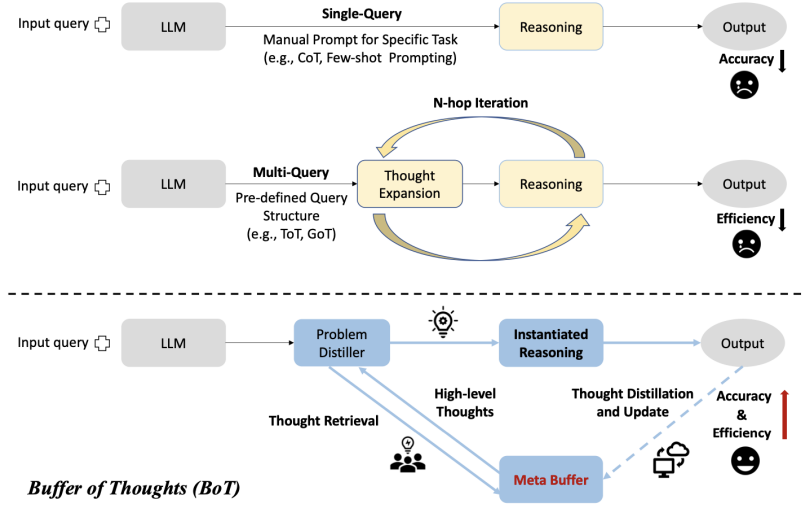


Figure 16: Chain-of-Thought, Self-Consistency, and Buffer of Thoughts [Yang et al., 2024]

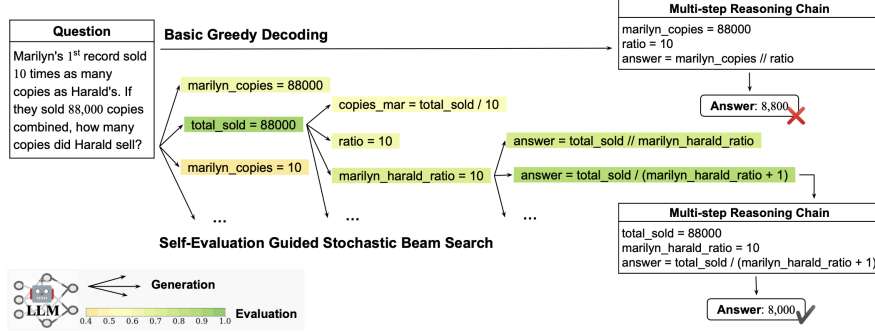


Figure 17: Self-evaluation in multi-step reasoning in Beam-Search [Xie et al., 2024]

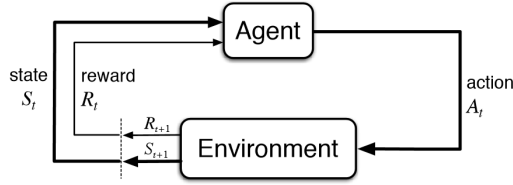


Figure 18: Reinforcement Learning [Sutton and Barto, 2018]

reasoning problems, this space can be very large. Beam-search solves this challenge by searching only a promising part of this space. It uses self-evaluation to control exploration and to evaluate (decode) reasoning steps. Figure 17 shows how Beam-search self-evaluation is used in multi-step reasoning. Beam search uses Program-aided-language models for math word problems [Gao et al., 2023]. Using a Codex backbone [Chen et al., 2021], it surpasses the few-shot baselines by 6.34%, 9.56%, and 5.46% on the GSM8K, AQuA, and StrategyQA benchmarks, respectively.

Reinforcement learning Reinforcement learning (RL) methods are another step in the sophistication of optimization algorithms. RL learns by interactive sampling, improving its policy based on rewards from the environment [Sutton and Barto, 2018]. To use reinforcement learning, the reasoning problem must be formulated as a Markov Decision Process: the agent-algorithm creates a prompt (an *action*), to sample a step (t) and get an answer (*state*, *reward*) from the environment-model (see Figure 18). The answer can then be used to improve the prompt (next action), just like reinforcement learning uses rewards to improve its policy of best actions for each state. The approaches that use reinforcement learning do so in the form of an external algorithm. No prompt has been created that performs RL by itself.

Progressive-hint-prompting (PHP) uses reinforcement learning to interactively improve prompts [Zheng et al., 2023]. Figure 19 illustrates the approach. PHP is an

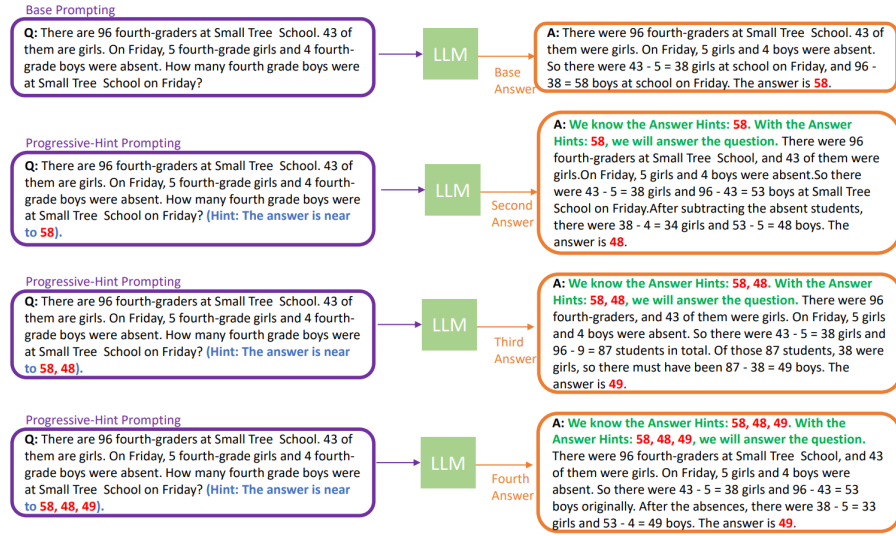


Figure 19: Progressive Hint Prompting [Zheng et al., 2023]

external algorithm that calls the LLM with dynamic prompts, using previously generated answers as hints to progressively prompt the LLM toward the correct answers. It works as follows: (1) given a question (prompt), the LLM provides a base answer, and (2) by combining the question and answer, the LLM is queried and we obtain a subsequent answer. We (3) repeat operation (2) until the answer becomes stable, like a regular policy-optimizing reinforcement learning algorithm. The authors have combined PHP with Chain-of-thought and with Self-consistency. Using GPT-4, state-of-the-art performance was achieved in grade school math questions (95%), simple math word problems (91%) and algebraic question answering (79%).

Another approach that is motivated by improving answers from feedback, is Self-refine [Madaan et al., 2023]. In this method, initial outputs from LLMs are improved through iterative feedback and refinement. Like PHP, the LLM generates an initial output and provides feedback for its answer, using it to refine itself, iteratively. Figures 20 and 21 illustrate the approach.

Self-refine prompts the LLM in three ways: (1) for initial generation, (2) for feedback, and (3) for refinement. Note that Self-refine follows a greedy reasoning chain, learning from feedback. Self-refine has been used with GPT-3.5 and GPT-4 as base LLMs, and has been benchmarked on dialogue response generation [Askari et al., 2024], code optimization, code readability improvement, math reasoning, sentiment reversal, acronym generation, and constrained generation, showing substantial improvements over the base models.

Another approach that combines reinforcement learning and LLMs is ReAct [Yao et al., 2022]. Most works so far have focused on reasoning by the LLM, not on actions by an agent. A key element of reinforcement learning is that it learns a policy for

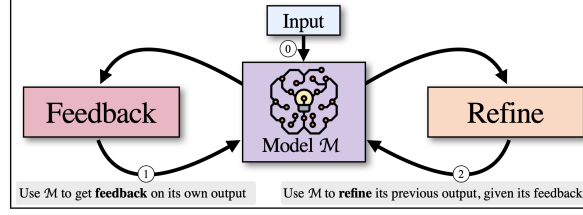


Figure 20: Self-Refine [Madaan et al., 2023]

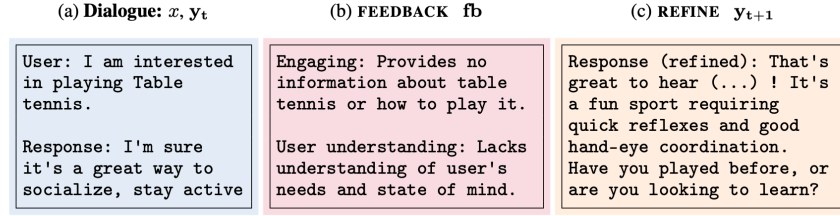


Figure 21: Self-Refine [Madaan et al., 2023]

an environment. The goal of ReAct is to combine progress in reasoning with action plan generation. (Or, to put it differently, most approaches use RL to improve LLM-reasoning, ReAct uses LLMs to improve RL agent policies.) ReAct uses Chain-of-thought prompt-learning as part of an RL framework that also uses external knowledge sources (Wikipedia) and finetuning, for error reduction, grounding, and for reducing hallucination. The framework allows hand-written prompts. Figure 22 shows four different prompting strategies. On two interactive decision making benchmarks (ALF-World and WebShop), ReAct outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively, while being prompted with one or two in-context examples.

The ReAct work has been developed further. Reflexion [Shinn et al., 2024] is built on top of ReAct. The goal is to create AI agents that learn by reflecting on failures and enhancing their results, much like humans do. Reflexion uses three language models: actor, evaluator, and reflector. It works as follows: (1) an actor generates text and actions, (2) an evaluator model scores the outputs produced by the actor, and (3) a self-reflection model generates verbal reinforcement cues to assist the actor to self-improve (see Figure 23). For the actor, Chain-of-thought [Wei et al., 2022b] and ReAct [Yao et al., 2022] can be used. Reflexion is evaluated on decision-making, reasoning, and coding tasks. Improvements of 10-20 percentage points are reported. Figure 24 shows three different prompting applications.

To conclude this overview of reinforcement learning approaches, we discuss an application in the games domain. Voyager [Wang et al., 2023] is an agent for the

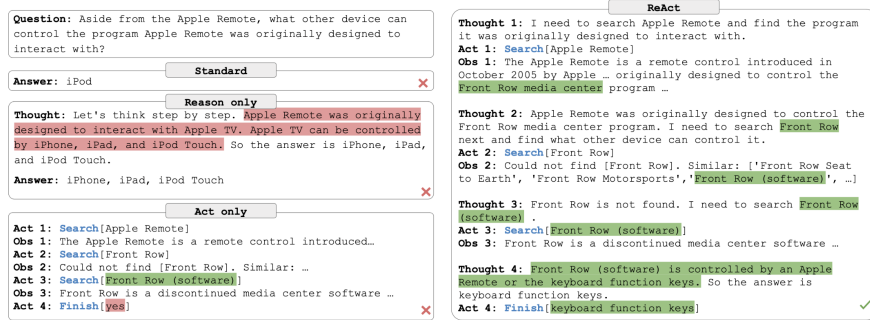


Figure 22: Comparison of four prompting strategies: Standard, Chain of Thought (reason), Action only, and ReAct [Yao et al., 2022]

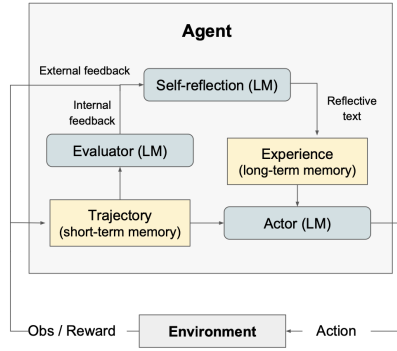


Figure 23: Architecture of Reflexion [Shinn et al., 2024]

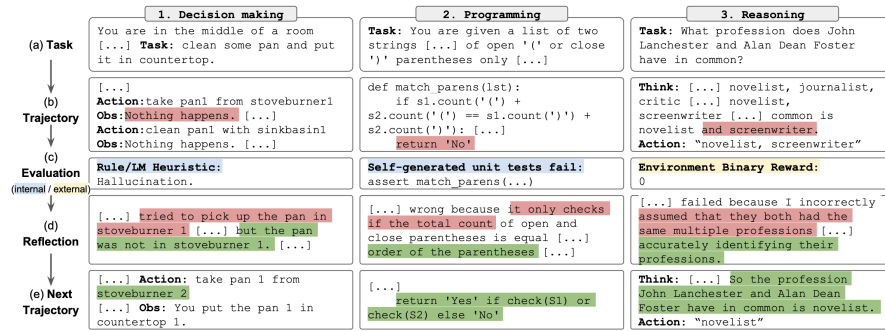


Figure 24: Comparison of three application areas [Shinn et al., 2024]

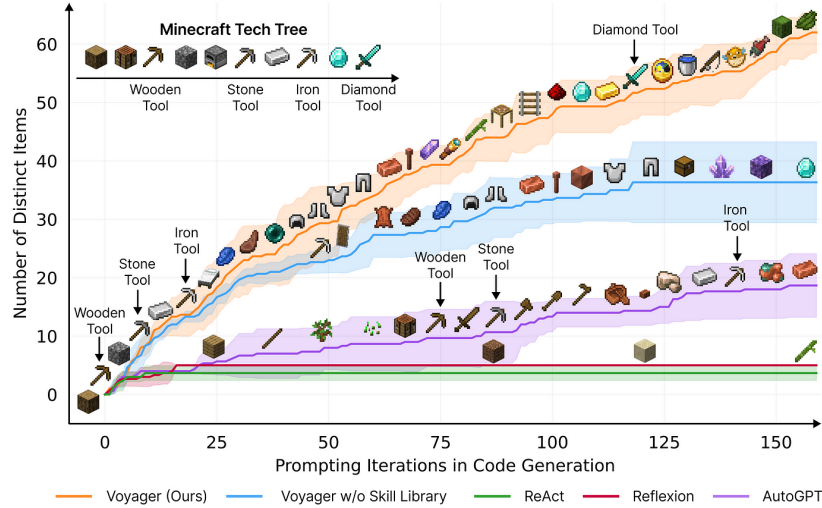


Figure 25: Performance of Voyager in Minecraft [Wang et al., 2023]

game of Minecraft that uses an iterative prompting mechanism that generates code for embodied control. The mechanism includes Self-verification [Shinn et al., 2024]. The agent has a skill library and an automatic curriculum to maximize exploration. Voyager interacts with GPT-4 through prompts. The goal of Voyager’s prompts is to discover as many diverse items in Minecraft as possible, a form of novelty search [Eysenbach et al., 2018]. Voyager performs well, it shows in-context lifelong learning capability and reaches high scores by acquiring many tools (see Figure 25).

6 Discussion

We have reviewed approaches for prompt-based reasoning by LLMs, highlighting techniques that have achieved a breakthrough in reasoning performance. It is time for reflection on limitations in the approaches, suggesting promising areas of future work. First we discuss issues concerning hallucination, faithful reasoning, and scaling. Then we discuss what LLMs can and cannot do. Then, we highlight connections with sequential decision processes and metacognition, and end with a research agenda.

6.1 Hallucination, Faithfulness and Scaling

Most works on reasoning in LLMs are experimental in nature. The success of in-context learning and Chain-of-thought reasoning is attracting the attention of work providing deeper insight into the reasoning processes in language models.

Saparov and He [2022] introduce a synthetic question/answer dataset designed to evaluate the reasoning abilities of LLMs. The work showed that LLMs are capable of

reasoning to a certain degree, but that Chain-of-thought struggles with proof trees with a wide branching factor. In another study, Wang et al. [2022a] also aim to increase our understanding of how Chain-of-thought works. The authors find that it continues to work even with invalid steps in the reasoning chain. They also find that the order of the reasoning steps is important for good results. Prompts should be relevant to the question, and coherent (steps should be in the correct order). Jin et al. [2024] study the impact of reasoning step length on LLMs, finding a strong positive correlation between the length of the prompt and reasoning abilities.

These works highlight ways in which LLM-reasoning can see things that are not there. Next, we discuss works on failure modes of the Chain-of-thought approach, studying whether the reasoning of the LLM is faithful, or that it gives the right answer for the wrong reason.

6.1.1 Faithfulness

Chain-of-thought and other approaches prompt a language model to take certain steps to solve the problem that the prompt specifies. One can ask the question, whether those steps are indeed the steps that the model has followed (faithful reasoning) or whether it took another road to arrive at the correct answer (unfaithful reasoning). A few studies measure the faithfulness of reasoning by LLMs. Lanham et al. [2023] notes that just like organic reasoners, a model’s reasoning may be post-hoc, it may be constructed after a certain conclusion has been found. By deliberately adding mistakes to the chain of thought, the authors measure the faithfulness of the model. They find a wide variation of post-hoc reasoning, with a tendency of larger models to be less faithful. Like regular LLMs, when not properly grounded, (Chain-of-thought) reasoning suffers from hallucination.

Another study adds deliberate bias to the prompt. For example, in a multiple-choice setting, they always make answer (a) the correct answer [Turpin et al., 2024]. They find that a bias towards wrong answers can cause significant drops in accuracy, and that models frequently generate Chain-of-thought explanations rationalizing wrong answers. The authors further note that, insofar as language models are trained on human-written explanations, that explanations may be incomplete or wrong. Human explanations may omit crucial steps of the causal chain, may provide an unfaithful account of the human reasoning process, or may be aimed at convincing others, instead of providing the true causes of a decision.

To address issues of faithfulness, Lyu et al. [2023] propose Faithful-chain-of-thought. This approach involves two stages. First, the natural language query is translated into a formal symbolic language. Second, the problem-solving stage processes the formal language, and can explain the reasoning steps it has thus taken. For the symbolic language, Python, Datalog, or PDDL is suggested. Faithfulness studies tell us more about how models reason. Further surveys on this topic are Mondorf and Plank [2024], Chuang et al. [2024], Luo et al. [2023], Paul et al. [2024],

6.1.2 Scaling

The emergent abilities of LLMs have prompted research into the nature of scaling and reasoning with LLMs, and, specifically, how reasoning capabilities can be transferred to smaller language models. Scaling laws of LLMs are an active area of study, see for example Kaplan et al. [2020], Henighan et al. [2020], Hoffmann et al. [2022]. Given the computational cost of LLMs, there is much interest in transferring knowledge to small language models. Comprehensive surveys on knowledge distillation are Xu et al. [2024], Gu et al. [2023]. For reasoning specifically, Magister et al. [2022] have studied reasoning in small language models, using a student model that learns from a teacher model, by finetuning. Another study related to Self-taught-reasoner [Li et al., 2022a] focuses on explanation in small language models, achieving similar results.

Other works focus on prompt distillation for retrieval Dai et al. [2022], recommendation [Li et al., 2023], distillation to embodied agents of Chain-of-thought reasoning [Choi et al.], and distillation of LLM graph reasoning [Zhang et al., 2024]. Distillation of reasoning to smaller models can work surprisingly well in situations with more explicit instructions. Distillation is also proposed for bringing results of System 2 reasoning to System 1 Yu et al. [2024], which brings us to the topic of metacognition (see Section 6.2.3).

6.2 Limitations: What LLMs Can and Cannot do

The capabilities of LLMs are impressive. LLMs can be seen as large text-based surrogate models of the world (or the world how we describe it on the internet), and thus allow us to reason in a way that we can understand about a large variety of contexts and problems. Reasoning tasks, such as math word problems, were one of the capabilities that LLMs could not achieve, until recently. Let us look more closely at what language models can and cannot do.

6.2.1 What Can LLMs Do?

With the right prompt LLMs are able to solve many of the problems in reasoning grade school math word benchmarks. Prompt-based learning is able to perform *reasoning* tasks such as math word problems, robotic movement, and Python code generation, at inference time, without expensive parameter training.

We note that a simple taxonomy of generate-evaluate-control is able to describe the structure of the current LLM reasoning literature well. Furthermore, the accuracy of the reasoning chains can be improved with ensemble methods, or self-verification. Hallucination can be reduced by grounding the model with external models, such as for robotic affordances, and information retrieval from search engines and Wikipedia. Going a step further, using external control algorithms (such as search or RL) as scaffolding, dynamic prompts can use the LLMs to perform complex and interactive reasoning patterns.

Note that the reasoning control is now two layers away from the core LLM: an external control algorithm, on top of in-context-learning, dynamically generating prompts for the LLM. This is reasoning *with* prompts *with* LLMs, not *by*.

At this point, it is interesting to note the confluence of the two schools of classical artificial intelligence (AI), symbolic and connectionist.⁵ Search and reinforcement learning are rooted in the symbolic AI tradition, while LLMs are rooted in the connectionist tradition. The literature in this survey combines the two traditions. High performance reasoning is created with a (symbolic) searcher/learner on top of a (connectionist) LLM. In other fields similar combinations can be seen (for example, AlphaFold Bryant et al. [2022], Jumper et al. [2021] and retrosynthesis of molecules Segler et al. [2018]). The LLM helps ground symbolic reasoning methods in language; symbolic methods help create prompts that let the LLM perform reasoning. How the two traditions will continue to improve each other, we will see in further research.

We note that benchmarks such as GSM8K have been central for the progress in the field, and that while reasoning started with math word problems, the field has extended to robotics, autonomous agents, games, and most emphatically computer code. Formal languages play an important role in the intermediate multi-step reasoning chains.

A side effect from the work on reasoning is the emergence of a new few-shot learning approach for sequential decision-making processes (SDP)[Littman, 1996]. Traditionally these processes are solved with reinforcement learning (such as DQN Mnih et al. [2015], PPO [Schulman et al., 2017] and SAC Haarnoja et al. [2018]), achieving good results, but suffering from high sample complexity for larger problems Plaet et al. [2023]. The emergence of few-shot in-context learning for solving SDPs opens a research avenue to find out what SDPs few-shot prompt-learning will be able to solve.

6.2.2 What Can LLMs Not Do?

Now that grade school math word problems are largely solvable, harder reasoning benchmarks in other domains are appearing [Ahn et al., 2024]. Another line of research argues that LLMs cannot reason, providing examples where LLMs fail, and discussing potential reasons. Berglund et al. [2023] show that LLMs can fail to generalize in surprising ways. They provide the example that if a model is trained to report that "Valentina Tereshkova was the first woman to travel to space", it will not automatically be able to answer the question, "Who was the first woman to travel to space?" pointing to a lack in semantic understanding of LLMs. Other work suggests that results are less generalizable and transferable than often assumed, showing how base-10 arithmetic skills do not transfer to base-9 arithmetic problems Wu et al. [2024]. The question which problems LLMs can and cannot solve will continue to motivate researchers.

Other works study the dangers of the size of LLMs. Bender et al. [2021] mention the environmental risks associated with the large computational training demands, as well as the difficulty of understanding the training data, for example in the context of bias. Furthermore, there are ethical, legal, and copyright concerns regarding the data that LLMs are trained on. Finally, to prevent putting too much trust in the outcome

⁵Reasoning and planning have been studied since the start of artificial intelligence, starting with logic and reasoning [Newell and Simon, 1961], search algorithms in puzzles and board games [Korf, 1999, Plaet, 2020], robot planning [Fikes and Nilsson, 1971], classical machine learning such as decision trees and support vector machines [Flach, 2012, Breiman, 2001, Cortes and Vapnik, 1995], through knowledge representation and the semantic web [Van Harmelen et al., 2008]. Ever since the success of the connectionist approach LeCun et al. [2015], Goodfellow et al. [2016] (deep learning, including LLMs) researchers have tried to join the two approaches.

of LLMs, we should understand their failure modes better, such as the well-publicized problems of hallucination (inventing facts that look right but are not).

Most of the reasoning capabilities exhibited by LLMs are due to the great representational powers of the transformer architecture, and how in-context learning is able to harness them. Prompt engineering and prompt control play a crucial role in the kind of reasoning that we have seen in the papers. Models can be instructed to write their own reasoning prompts; however, such Auto-GPT or Auto-CoT prompts need evaluation, verification, and grounding in the real world, to prevent degeneration into a hallucinatory world of their own. Models can also be instructed to interact with the world, and become the tool of external scaffolding that evaluates, controls and improves the prompts. Some of what we experience as reasoning *by* the LLM, is controlled by the prompt or the scaffolding algorithm. It is an open question if prompt learning is able to get the LLM to create a prompt to exhibit non-trivial reasoning by itself.

From the symbolic planning field there is also a critical view on the reasoning and planning abilities of LLMs [Valmeekam et al., 2023] giving examples of planning failures. They argue that LLMs can be used instead to improve heuristic elements of traditional planners, such as PDDL [Kambhampati et al., 2024], to strengthen traditional symbolic planning approaches.

Some of the names of the approaches surveyed in this paper are suggestive of self-awareness and self-reflective capabilities. True self-reflection, or metacognition, is still largely outside the capabilities of current LLMs. LLMs can be prompted to reason, to take small steps, to self-evaluate, and their search process can be controlled by an external algorithm. The self-reflective type of “intelligence” is written into the prompt by the prompt engineer or the interactive algorithm. We are unaware of any LLM that has been made to reflect on, or even control, its reasoning processes, controlling how many reasoning steps it should take, or limiting its reasoning once the answer had become good enough. True self-reflection remains future work, although some steps have been taken, as we will discuss next.

6.2.3 Reasoning towards Metacognition

Human thought exhibits the ability to reason about self, we are able to think about our own thinking processes. Metacognition studies these topics [Veenman et al., 2006]. Prompted by the success of Chain-of-thought and the works that we have surveyed, metacognition has also been studied in the context of LLMs [Toy et al., 2024].

Many reasoning approaches highlight self-reflective aspects in their names and in how they work. The prompts that prompt the models to reason are being improved with the outcome of the reasoning process, and in Buffer-of-thoughts thought-templates are used that are derived from other reasoning processes. Wang and Zhao [2023] study Metacognitive-prompting. Inspired by Chain-of-thought and Self-consistency, they create manually designed prompts to increase the understanding of language models. Figure 26 illustrates the relation between metacognitive human thought processes and metacognitive LLM prompting. Another work, again inspired by Chain-of-thought and Self-consistency, connects psychology and LLMs. Didolkar et al. [2024] study metacognitive capabilities of LLMs in mathematical problem solving, both on GSM8K and on the harder MATH problems [Hendrycks et al., 2021]. First, the model is

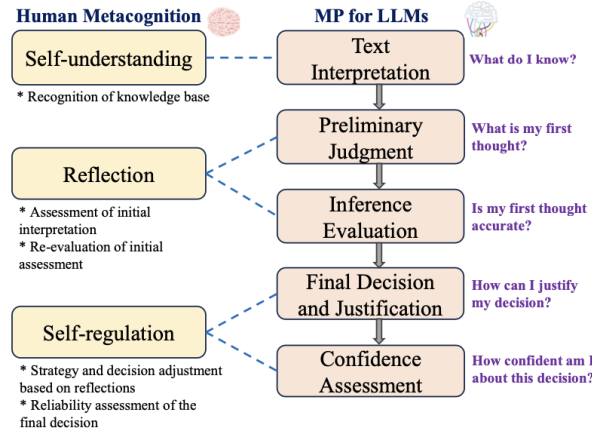


Figure 26: Metacognitive Prompting and the link with human metacognitive processes [Wang and Zhao, 2023]

prompted to find a skill name for each problem instance in the dataset. For 7000 instances of GSM8K, 500 skill names were found by the model. Next, these 500 names are clustered down to 22 skills. They find that by using the names of these 22 skills in Chain-of-thought-like prompts, more problems are solved than with standard Chain-of-Thought/Self-consistency/PAL prompts. Examples of the 22 skill names are *multiplication-and-addition*, *basic-arithmetic*, *subtraction*, and *algebra*. Interestingly, the authors find that the skill exemplar repository that is trained on a strong model (GPT-4), also down-translates to a weak model (GPT-3). The performance of the weak model benefits from the skill-name-enhanced prompts.

The connection between reasoning in LLMs and full-blown metacognitive reasoning is in its early stages. Exciting future research may appear.

6.3 Research Agenda

At the end of this discussion, we present promising topics for future work. Reasoning with LLMs is an active field of research. It brings together elements of symbolic reasoning, connectionism, natural language, autonomous agents, and affective reasoning [Broekens et al., 2023] with the promise of artificial general intelligence.

For the future, the surveyed works point in the following directions. First we discuss topics for the field of LLM-reasoning itself, then we discuss more general machine learning topics that are important for progress in LLM-reasoning, and finally we discuss more longer term, fundamental topics.

Specific research topics for reasoning with LLMs are:

- *Control and prompt-learning*—Search control beyond greedy search is implemented as an external algorithm. Is it possible to incorporate all stages of the

reasoning pipeline into an interactive prompt? Can we make a prompt that performs dynamic search-like step control without external scaffolding?

- *Code*—Progress in reasoning using formal languages and computer code has been quite promising. GitHub Copilot is a success. Further integration of LLM-reasoning with software engineering tools is a promising area of research that can have a large practical impact on how software is written.
- *Grounding*—Reasoning in LLMs has been successfully applied in autonomous agents, robotics, and games. A challenge is the grounding of the reasoning process in the environment. How can we help LLMs to actively find new information when the reasoning outcome is uncertain? Is retrieval augmented generation the future? Is the future of the reasoning-LLM a search engine [Verberne, 2024]?

Generic topics in machine learning that also influence prompt-based reasoning research are:

- *Benchmarks*—Progress in LLMs is governed by the availability of the right benchmarks. The current favorite is GSM8K, for grade school math. As the field progresses, other benchmarks will become prevalent: benchmarks with more difficult tasks, and benchmarks for other applications in autonomous agents and robotics.
- *Faithfulness*—Our theoretical understanding of prompt-based reasoning with LLMs is incomplete. The research on faithfulness highlights one example of our lack of understanding. In general, more insight into the working of multi-step in-context learning in LLMs is dearly needed.
- *Small language models*—Efficiency is an important element for wide adoption of language models. Important topics are distillation of reasoning to small language models and an understanding of scaling laws.
- *Few-shot Reinforcement Learning*—Small reasoning problems can be solved with few-shot in-context learning. Can we solve larger sequential decision processes, reducing the sample complexity in reinforcement learning?

For longer term future work, the following more fundamental questions are important:

- *Symbolic and Connectionist Computation*—How can we further improve LLM-reasoning: how can LLMs benefit from symbolic reasoning prompts and how can LLMs help ground symbolic reasoning in language?
- *Metacognition*—Much of the research into reasoning guides the model how it should solve a problem. Is it helpful to introduce named concepts for different kinds of reasoning? Can the model find these concepts by itself? Making the LLM “think” step by step is a first step towards influencing the model’s own “thought” processes. The first works on LLM metacognition have appeared, and artificial general intelligence will pursue this further.

7 Conclusion

Prompt-based in-context learning is an efficient machine learning method, requiring no parameter updates to the LLM. While achieving good performance on language tasks (System 1), performance on reasoning tasks (System 2) was lacking. Reasoning tasks, such as math word problems, are typically solved in step-by-step fashion. Recently prompts have been developed that guide an LLM to “think step by step” (Chain-of-thought), and to evaluate and verify the step results. The performance of reasoning with LLMs has improved greatly. Together, the surveyed methods allow the LLM to follow high-quality multi-step reasoning chains. Python code or other formal languages have been used successfully to reduce the error in reasoning steps. Also, in the field of autonomous agents and robotic action, good performance has been achieved by grounding reasoning answers in the environment and the physical constraints of robotic movement.

For complex reasoning tasks a large number of reasoning steps may be generated. To control the size of the reasoning space interactively, external scaffolding algorithms can be used. Often, variations on search algorithms or reinforcement learning are used. The symbolic and connectionist AI traditions come together in reasoning prompts and search algorithms that help LLM neural networks solve natural language math word and related problems.

Among the most popular reasoning benchmarks in this survey is GSM8K, which contains 8500 grade school math word problems. With LLMs such as GPT-3, reasoning approaches show an improvement of 20-50% points over standard prompting methods. For further progress in the field, the development of other challenging benchmarks is important.

The field of reasoning with LLMs is quite new, and theoretical understanding is lacking in important areas, such as faithful reasoning (models may sometimes find the right answer for the wrong reason). Although prompt-based learning allows few-shot learning, the computational needs of LLMs pretraining and finetuning are still high, hence the interest in small language models. Reasoning skills that work in large models can often be transferred to small models.

Human thought is capable of metacognition, we can think about our thinking process. Many of the names of the approaches in this survey suggest a link to metacognition (Reflexion, Self-refine, Self-improvement, Inner-monologue). The first preliminary experiments of language models that reason about their reasoning skills have appeared.

LLM-reasoning is an active field of research, with connections to artificial general intelligence. The field has shown great progress. Based on current limitations and open questions we provide a research agenda highlighting opportunities for further progress in harder reasoning problems, metacognition, and small language models, amongst others.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren-
cia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal
Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. Large lan-
guage models for mathematical reasoning: Progresses and challenges. *arXiv preprint
arXiv:2402.00157*, 2024.
- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron
David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al.
Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint
arXiv:2204.01691*, 2022.
- Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Han-
naneh Hajishirzi. Mathqa: Towards interpretable math word problem solving with
operation-based formalisms. *arXiv preprint arXiv:1905.13319*, 2019.
- Arian Askari, Roxana Petcu, Chuan Meng, Mohammad Aliannejadi, Amin Abol-
ghasemi, Evangelos Kanoulas, and Suzan Verberne. Self-seeding and multi-intent
self-instructing llms for generating intent-aware information-seeking dialogs. *arXiv
preprint arXiv:2402.11633*, 2024.
- Richard Bellman. Dynamic programming. *science*, 153(3731):34–37, 1966.
- Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret
Shmitchell. On the dangers of stochastic parrots: Can language models be too big?
In *Proceedings of the 2021 ACM conference on fairness, accountability, and trans-
parency*, pages 610–623, 2021.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum
learning. In *Proceedings of the 26th annual international conference on machine
learning*, pages 41–48, 2009.
- Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland,
Tomasz Korbak, and Owain Evans. The reversal curse: Llms trained on” a is b” fail
to learn” b is a”. *arXiv preprint arXiv:2309.12288*, 2023.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski,
Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr
Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large lan-
guage models. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
volume 38, pages 17682–17690, 2024.
- Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- Joost Broekens, Bernhard Hilpert, Suzan Verberne, Kim Baraka, Patrick Gebhard, and
Aske Plaat. Fine-grained affective processing capabilities emerging from large lan-
guage models. In *2023 11th Intl Conf on Affective Computing and Intelligent Inter-
action (ACII)*, pages 1–8. IEEE, 2023.

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Patrick Bryant, Gabriele Pozzati, and Arne Elofsson. Improved prediction of protein-protein interactions using alphafold2. *Nature communications*, 13(1):1265, 2022.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.
- Xinyun Chen, Chen Liang, Adams Wei Yu, Denny Zhou, Dawn Song, and Quoc V Le. Neural symbolic reader: Scalable integration of distributed and symbolic representations for reading comprehension. In *International Conference on Learning Representations*, 2019.
- Xinyun Chen, Maxwell Lin, Nathan Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.
- Ting-Rui Chiang and Yun-Nung Chen. Semantically-aligned equation generation for solving and reasoning math word problems. *arXiv preprint arXiv:1811.00720*, 2018.
- Wonje Choi, Woo Kyung Kim, Minjong Yoo, and Honguk Woo. Embodied cot distillation from llm to off-the-shelf agents. In *Forty-first International Conference on Machine Learning*.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- Yu-Neng Chuang, Guanchu Wang, Chia-Yuan Chang, Ruixiang Tang, Fan Yang, Mengnan Du, Xuanning Cai, and Xia Hu. Large language models as faithful explainers. *arXiv preprint arXiv:2402.04678*, 2024.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20: 273–297, 1995.

- Zhuyun Dai, Vincent Y Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B Hall, and Ming-Wei Chang. Promptagator: Few-shot dense retrieval from 8 examples. *arXiv preprint arXiv:2209.11755*, 2022.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Aniket Didolkar, Anirudh Goyal, Nan Rosemary Ke, Siyuan Guo, Michal Valko, Timothy Lillicrap, Danilo Rezende, Yoshua Bengio, Michael Mozer, and Sanjeev Arora. Metacognitive capabilities of llms: An exploration in mathematical problem solving. *arXiv preprint arXiv:2405.12205*, 2024.
- Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. A survey on in-context learning. *arXiv preprint arXiv:2301.00234*, 2022.
- John Dunlosky and Janet Metcalfe. *Metacognition*. Sage Publications, 2008.
- Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*, 2018.
- Richard E Fikes and Nils J Nilsson. Strips: A new approach to the application of theorem proving to problem solving. *Artificial intelligence*, 2(3-4):189–208, 1971.
- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.
- Peter Flach. *Machine learning: the art and science of algorithms that make sense of data*. Cambridge university press, 2012.
- Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. Complexity-based prompting for multi-step reasoning. In *The Eleventh International Conference on Learning Representations*, 2022.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799. PMLR, 2023.
- Louie Giray. Prompt engineering with chatgpt: a guide for academic writers. *Annals of biomedical engineering*, 51(12):2629–2633, 2023.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2023.

- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Tom Henighan, Jared Kaplan, Mor Katz, Mark Chen, Christopher Hesse, Jacob Jackson, Heewoo Jun, Tom B Brown, Prafulla Dhariwal, Scott Gray, et al. Scaling laws for autoregressive generative modeling. *arXiv preprint arXiv:2010.14701*, 2020.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(9):5149–5169, 2021.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. *arXiv preprint arXiv:2210.11610*, 2022a.
- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.
- Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022b.
- Mike Huisman, Jan N Van Rijn, and Aske Plaat. A survey of deep meta-learning. *Artificial Intelligence Review*, 54(6):4483–4541, 2021.
- Shima Imani, Liang Du, and Harsh Shrivastava. Mathprompter: Mathematical reasoning using large language models. *arXiv preprint arXiv:2303.05398*, 2023.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018.

- Mingyu Jin, Qinkai Yu, Haiyan Zhao, Wenyue Hua, Yanda Meng, Yongfeng Zhang, Mengnan Du, et al. The impact of reasoning step length on large language models. *arXiv preprint arXiv:2401.04925*, 2024.
- John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021.
- Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 4:237–285, 1996.
- Daniel Kahneman. *Thinking, fast and slow*. macmillan, 2011.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Kaya Stechly, Mudit Verma, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Llms can’t plan, but can help planning in llm-modulo frameworks. *arXiv preprint arXiv:2402.01817*, 2024.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Tom Kocmi, Rachel Bawden, Ondřej Bojar, Anton Dvorkovich, Christian Federmann, Mark Fishel, Thamme Gowda, Yvette Graham, Roman Grundkiewicz, Barry Haddow, et al. Findings of the 2022 conference on machine translation (wmt22). In *Proceedings of the Seventh Conference on Machine Translation (WMT)*, pages 1–45, 2022.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. Mawps: A math word problem repository. In *2016 conference of the north american chapter of the association for computational linguistics: human language technologies*, pages 1152–1157, 2016.
- Richard E Korf. Artificial intelligence search algorithms, 1999.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
- Lei Li, Yongfeng Zhang, and Li Chen. Prompt distillation for efficient llm-based recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 1348–1357, 2023.

- Shiyang Li, Jianshu Chen, Yelong Shen, Zhiyu Chen, Xinlu Zhang, Zekun Li, Hong Wang, Jing Qian, Baolin Peng, Yi Mao, et al. Explanations from large language models make small reasoners better. *arXiv preprint arXiv:2210.06726*, 2022a.
- Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. On the advance of making language models better reasoners. *arXiv preprint arXiv:2206.02336*, 2022b.
- Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. *arXiv preprint arXiv:1705.04146*, 2017.
- Michael Lederman Littman. *Algorithms for sequential decision-making*. Brown University, 1996.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.
- Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. *arXiv preprint arXiv:2310.01061*, 2023.
- Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. Faithful chain-of-thought reasoning. *arXiv preprint arXiv:2301.13379*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.
- Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. Teaching small language models to reason. *arXiv preprint arXiv:2212.08410*, 2022.
- Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing english math word problem solvers. *arXiv preprint arXiv:2106.15772*, 2021.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.
- Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey. *arXiv preprint arXiv:2402.06196*, 2024.

- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- Philipp Mondorf and Barbara Plank. Beyond accuracy: Evaluating the reasoning behavior of large language models—a survey. *arXiv preprint arXiv:2404.01869*, 2024.
- Shashi Narayan, Shay B Cohen, and Mirella Lapata. Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *arXiv preprint arXiv:1808.08745*, 2018.
- Allen Newell and Herbert A Simon. Computer simulation of human thinking: A theory of problem solving expressed as a computer program permits simulation of thinking processes. *Science*, 134(3495):2011–2017, 1961.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031*, 2016.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are nlp models really able to solve simple math word problems? *arXiv preprint arXiv:2103.07191*, 2021.
- Debjit Paul, Mete Ismayilzada, Maxime Peyrard, Beatriz Borges, Antoine Bosselut, Robert West, and Boi Faltings. Refiner: Reasoning feedback on intermediate representations. *arXiv preprint arXiv:2304.01904*, 2023.
- Debjit Paul, Robert West, Antoine Bosselut, and Boi Faltings. Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning. *arXiv preprint arXiv:2402.13950*, 2024.
- Aske Plaat. *Learning to play: reinforcement learning and games*. Springer Nature, 2020.
- Aske Plaat. *Deep reinforcement learning*. Springer, Singapore, 2022.

- Aske Plaat, Walter Kusters, and Mike Preuss. High-accuracy model-based reinforcement learning, a survey. *Artificial Intelligence Review*, 56(9):9541–9573, 2023.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Subhro Roy and Dan Roth. Solving general arithmetic word problems. *arXiv preprint arXiv:1608.01413*, 2016.
- Omer Sagi and Lior Rokach. Ensemble learning: A survey. *Wiley interdisciplinary reviews: data mining and knowledge discovery*, 8(4):e1249, 2018.
- Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024.
- Abulhair Saparov and He He. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. *arXiv preprint arXiv:2210.01240*, 2022.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Marwin HS Segler, Mike Preuss, and Mark P Waller. Planning chemical syntheses with deep neural networks and symbolic ai. *Nature*, 555(7698):604–610, 2018.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving neural machine translation models with monolingual data. *arXiv preprint arXiv:1511.06709*, 2015.

- Jianhao Shen, Yichun Yin, Lin Li, Lifeng Shang, Xin Jiang, Ming Zhang, and Qun Liu. Generate & rank: A multi-task framework for math word problems. *arXiv preprint arXiv:2109.03034*, 2021.
- Zhengliang Shi, Shen Gao, Xiuyi Chen, Yue Feng, Lingyong Yan, Haibo Shi, Dawei Yin, Zhumin Chen, Suzan Verberne, and Zhaochun Ren. Chain of tools: Large language model is an automatic multi-tool learner. *arXiv preprint arXiv:2405.16533*, 2024a.
- Zhengliang Shi, Shen Gao, Xiuyi Chen, Lingyong Yan, Haibo Shi, Dawei Yin, Zhumin Chen, Pengjie Ren, Suzan Verberne, and Zhaochun Ren. Learning to use tools via cooperative and interactive agents. *arXiv preprint arXiv:2403.03031*, 2024b.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pages 894–906. PMLR, 2022.
- Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3:9–44, 1988.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. 2018.
- Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv preprint arXiv:1811.00937*, 2018.
- Yiming Tan, Dehai Min, Yu Li, Wenbo Li, Nan Hu, Yongrui Chen, and Guilin Qi. Can ChatGPT replace traditional KBQA models? An in-depth analysis of the question answering performance of the GPT LLM family. In *International Semantic Web Conference*, pages 348–367. Springer, 2023.
- Yi Tay, Mostafa Dehghani, Vinh Q Tran, Xavier Garcia, Jason Wei, Xuezhi Wang, Hyung Won Chung, Siamak Shakeri, Dara Bahri, Tal Schuster, et al. U12: Unifying language learning paradigms. *arXiv preprint arXiv:2205.05131*, 2022.
- Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.
- Jason Toy, Josh MacAdam, and Phil Tabor. Metacognition is all you need? using introspection in generative agents to improve goal-directed behavior. *arXiv preprint arXiv:2401.10910*, 2024.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don’t always say what they think: unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36, 2024.

- Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models-a critical investigation. *Advances in Neural Information Processing Systems*, 36:75993–76005, 2023.
- Frank Van Harmelen, Vladimir Lifschitz, and Bruce Porter. *Handbook of knowledge representation*. Elsevier, 2008.
- Niki van Stein and Thomas Bäck. Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics. *arXiv preprint arXiv:2405.20132*, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Marcel VJ Veenman, Bernadette HAM Van Hout-Wolters, and Peter Afflerbach. Metacognition and learning: Conceptual and methodological considerations. *Metacognition and learning*, 1:3–14, 2006.
- Suzan Verberne. *Is the search engine of the future a chatbot?* Inaugural lecture, Leiden University, 2024.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.
- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. *Advances in neural information processing systems*, 32, 2019.
- Boshi Wang, Sewon Min, Xiang Deng, Jiaming Shen, You Wu, Luke Zettlemoyer, and Huan Sun. Towards understanding chain-of-thought prompting: An empirical study of what matters. *arXiv preprint arXiv:2212.10001*, 2022a.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022b.
- Yuqing Wang and Yun Zhao. Metacognitive prompting improves understanding in large language models. *arXiv preprint arXiv:2308.05342*, 2023.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022a.

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022b.
- Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. *arXiv preprint arXiv:2211.00053*, 2022.
- Yixuan Weng, Minjun Zhu, Fei Xia, Bin Li, Shizhu He, Shengping Liu, Bin Sun, Kang Liu, and Jun Zhao. Large language models are better reasoners with self-verification. *arXiv preprint arXiv:2212.09561*, 2022.
- Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyürek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. In *2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1819–1862, 2024.
- Yisheng Xiao, Lijun Wu, Junliang Guo, Juntao Li, Min Zhang, Tao Qin, and Tie-yan Liu. A survey on non-autoregressive generation for neural machine translation and beyond. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023a.
- Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, et al. Chain-of-experts: When llms meet complex operations research problems. In *12th International Conference on Learning Representations*, 2023b.
- Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, James Xu Zhao, Min-Yen Kan, Junxian He, and Michael Xie. Self-evaluation guided beam search for reasoning. *Advances in Neural Information Processing Systems*, 36, 2024.
- Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models. *arXiv preprint arXiv:2402.13116*, 2024.
- Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E Gonzalez, and Bin Cui. Buffer of thoughts: Thought-augmented reasoning with large language models. *arXiv preprint arXiv:2406.04271*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.

- Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling system 2 into system 1. *arXiv preprint arXiv:2407.06023*, 2024.
- Eric Zelikman, Jesse Mu, Noah D Goodman, and Yuhuai Tony Wu. Star: Self-taught reasoner bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Yizhuo Zhang, Heng Wang, Shangbin Feng, Zhaoxuan Tan, Xiaochuang Han, Tianxing He, and Yulia Tsvetkov. Can llm graph reasoning generalize beyond pattern memorization? *arXiv preprint arXiv:2406.15992*, 2024.
- Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. *arXiv preprint arXiv:2210.03493*, 2022.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.
- Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. Progressive-hint prompting improves reasoning in large language models. *arXiv preprint arXiv:2304.09797*, 2023.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2024.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.